기초 Unity 스터디 4강

UI와 비동기 프로그래밍, 그리고 그 외의 잡다한 것들

시작하기 전에...

- 소스 코드 확인하기: Github Gist (https://gist.github.com/kodugi/6863a3c120b35284d7c0aad5d308e101)

- 실습 및 강의 자료의 단계별 소스 코드는 위 링크에서 확인 가능

시작하기 전에...

- 본 스터디의 목적은 Unity의 기능들을 다뤄 보면서 익숙해지는 것도 있지만, Unity를 이용하여 게임 개발을 할 때 갖춰야 할 '프로그래머처럼 사고하는 방법'에 대해 배우는 것도 목적의 일부

- 그렇기에 Unity의 자잘한 기능 하나하나까지는 다 다루지 않을 수 있음
  - ex) 애니메이션, UI, 셰이더, 파티클 등 → 각 주제 하나별로 한 차시씩 설명해도 모자라지만 핵심적인 내용까지는 아님

- 필요한 태도: '이런 기능이 있으면 편할 것 같은데 없나?' 싶은 생각이 들 때마다 직접 찾아보자

- 한 가지 보완 방법: '아트를 위한 Unity 스터디' → Sprite Renderer, 애니메이션 등 Unity의 다양한 기능들이 간략하게나마 소개되어 있음

1. 지난 시간 마무리

지난 시간 실습 내용

- 정답

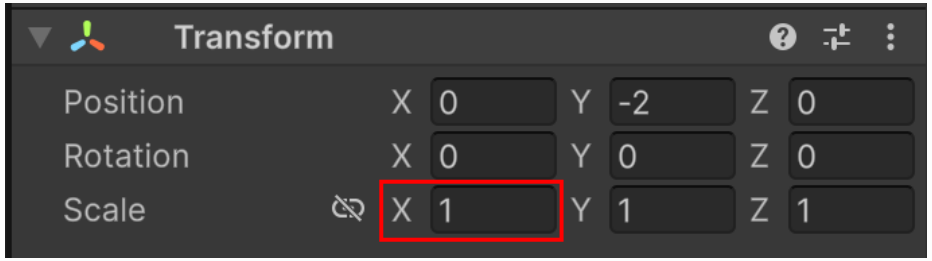
```
[SerializeField] private List<MapData> _mapDatas = new List<MapData>();
```

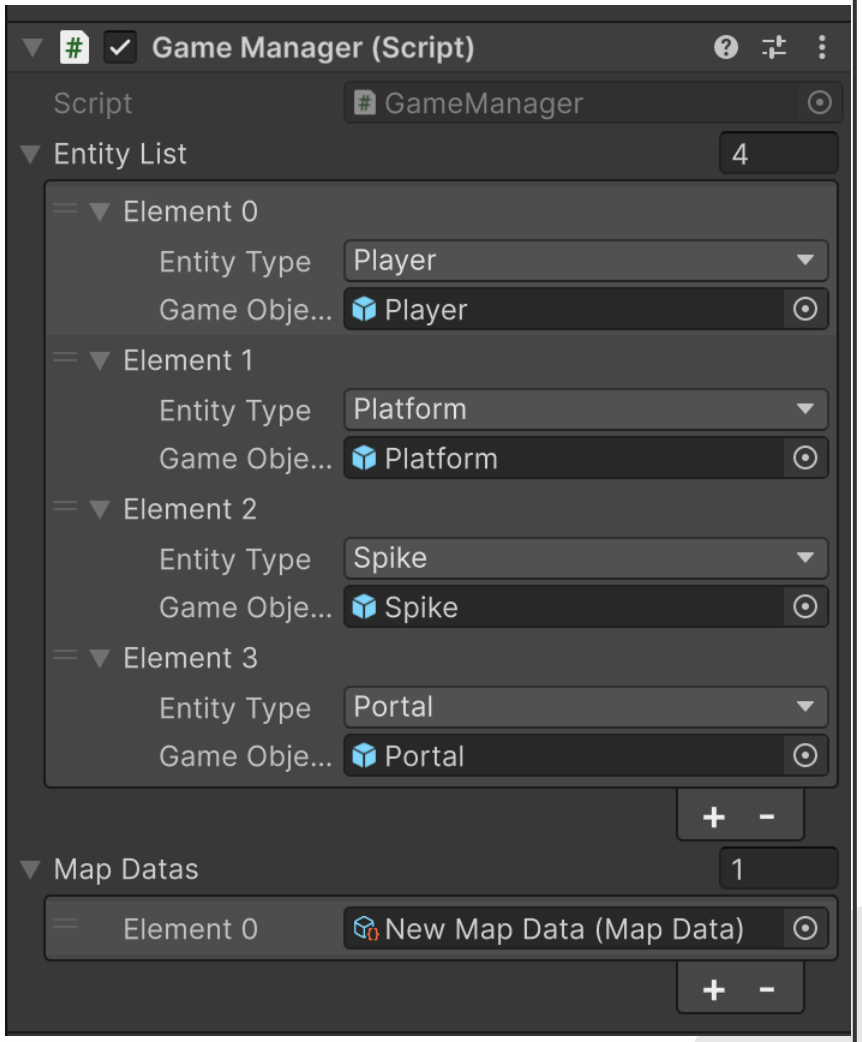
동적 생성 완성하기

- 지금까지 한 일
  - SerializeField로 프리팹 및 맵 데이터 받기
  - ScriptableObject로 맵 데이터 저장하기
- 이제 해야 할 일
  - GameManager에서 맵 데이터에 맞춰서 게임오브젝트들을 Instantiate하게 만들기

동적 생성 완성하기

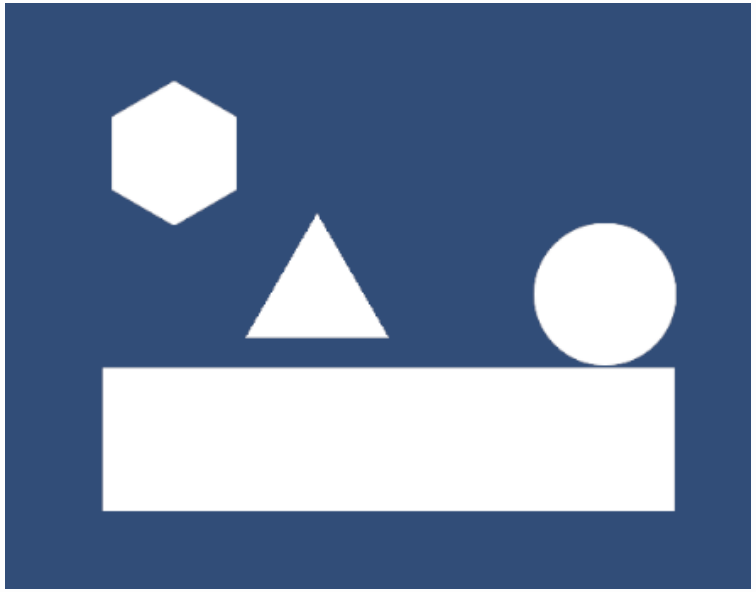
- 사전 준비
  - GameManager의 Game Manager 스크립트에서 Entity List 채우기(None은 제외)
  - GameManager의 Game Manager 스크립트에서 Map Datas에 Map Data 추가하기
  - 씬에서 GameManager를 제외한 우리가 생성한 모든 게임오브젝트 삭제하기
  - Platform 프리팹의 Transform에서 Scale의 x값을 1로 되돌리기

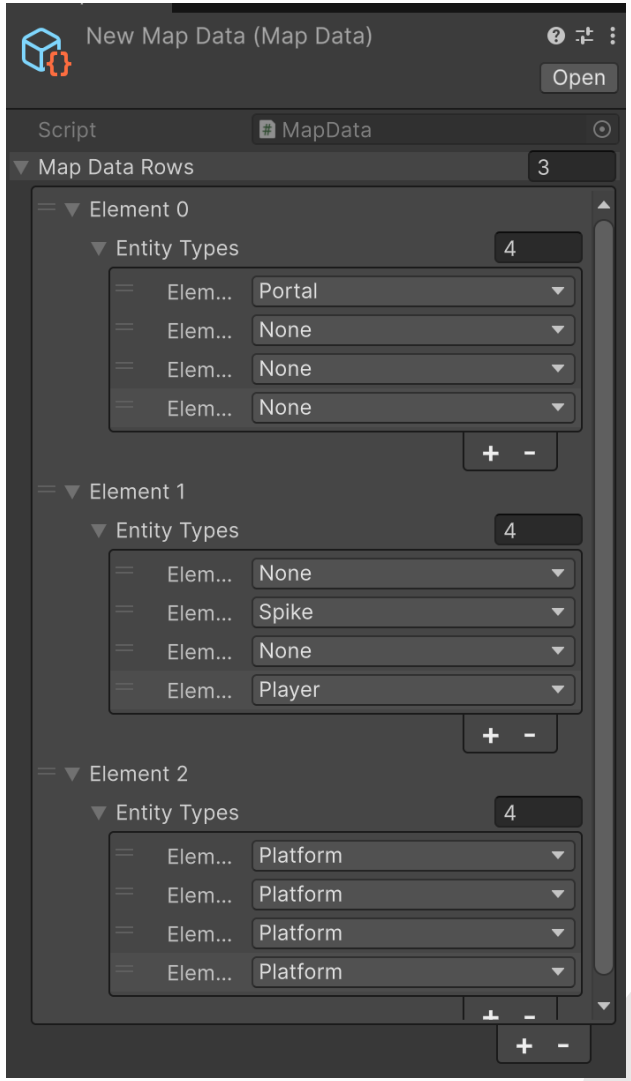




동적 생성 완성하기

- 테스트용 맵 데이터
- 생성된 모습





맵 로딩 구현하기

```
private void LoadMap(int idx)
{
    int height = _mapDatas[idx].mapDataRows.Count;
    for (int i = 0; i < height; i++)
    {
        int width = _mapDatas[idx].mapDataRows[i].entityTypes.Count;
        for (int j = 0; j < width; j++)
        {
            Vector3 coord = IndexToCoord( rowIdx: i,  colIdx: j, width, height);
            EntityType entityType = _mapDatas[idx].mapDataRows[i].entityTypes[j];
            if (entityType != EntityType.None)
            {
                Instantiate(_entityDict[entityType], coord, Quaternion.identity);
            }
        }
    }
}
```

Github Gist의 GameManager_1.cs 참조

실습 1: 끝난다는 것은 다시 시작된다는 것을

- 오브젝트를 생성한 것까지는 좋았는데, 오브젝트를 소멸시킬 필요도 있음
- 한 스테이지가 끝나고 다음 스테이지가 시작될 때, 생성된 모든 오브젝트를 삭제하고 다시 시작해야 하기 때문

- LoadMap으로 생성된 모든 오브젝트를 소멸시키는 RemoveLoadedMap 함수를 만들어보자

- 힌트
  - RemoveLoadedMap은 반환값도, 인수도 없으며, private 함수이다
  - LoadMap으로 생성된 오브젝트가 무엇인지를 어떤 식으로 저장하고 있다가, RemoveLoadedMap에서는 이 저장된 오브젝트들만 일일이 Destroy하면 되지 않을까?
  - LoadMap도 약간 수정해야 하는데, Instantiate가 게임오브젝트를 반환한다는 사실을 잊지 말자

실습 1: 끝난다는 것은 다시 시작된다는 것을

- 정답

```
private List<GameObject> _spawnedObjects = new List<GameObject>();

3 사용 위치
private void LoadMap(int idx)
{
    int height = _mapDatas[idx].mapDataRows.Count;
    for (int i = 0; i < height; i++)
    {
        int width = _mapDatas[idx].mapDataRows[i].entityTypes.Count;
        for (int j = 0; j < width; j++)
        {
            Vector3 coord = IndexToCoord( rowIdx: i,  colIdx: j, width, height);
            EntityType entityType = _mapDatas[idx].mapDataRows[i].entityTypes[j];
            if (entityType != EntityType.None)
            {
                _spawnedObjects.Add(Instantiate(_entityDict[entityType], coord, Quaternion.identity));
            }
        }
    }
}
```

```
private void DestroyLoadedMap()
{
    foreach (GameObject go in _spawnedObjects)
    {
        Destroy(go);
    }
    _spawnedObjects.Clear();
}
```

Github Gist의 GameManager_2.cs 참조

기나긴 여정 끝에...

- 다시 가시에 닿았을 때와 포탈에 닿았을 때에 대한 처리를 해 보자

- 맵 로딩은 GameManager가 담당하므로, GameManager에서 public 메서드를 제공해서 두 경우에 호출할 수 있게 하는 것이 자연스러움

- PlayerController에서 GameManager의 메서드를 어떻게 호출하는가?
  - GameObject.Find(string name): name과 같은 이름의 게임오브젝트를 하나 반환
  - Find로 찾은 오브젝트에서 GetComponent<GameManager>()를 호출하여 스크립트를 얻은 뒤, 이 스크립트에서 메서드를 호출하면 됨

GameManager 구현

```
private int _curIdx = 0;

// Start is called once before the first execution of Update after the MonoBehaviour is created
이벤트 함수
void Start()
{
    foreach(EntityDictEntry entityDictEntry in _entityList)
    {
        _entityDict[entityDictEntry.EntityType] = entityDictEntry.GameObject;
    }

    LoadMap( idx: 0);
    _curIdx = 0;
}
```

GameManager 구현

```
public void ResetMap()
{
    DestroyLoadedMap();
    LoadMap(_curIdx);
}

1 사용 위치
public void ToNextMap()
{
    DestroyLoadedMap();
    _curIdx = (_curIdx + 1) % _mapDatas.Count;
    LoadMap(_curIdx);
}
```

Github Gist의 GameManager_3.cs 참조

PlayerController 구현

```
void Start()
{
    _rb = GetComponent<Rigidbody2D>();
    _gameManager = GameObject.Find("GameManager").GetComponent<GameManager>();
}
```

```
private GameManager _gameManager;
```

이게 최선일까?
뒤에서 알아보자

PlayerController 구현

```
void OnTriggerEnter2D(Collider2D other)
{
    if (_gameManager == null)
    {
        return;
    }

    if (other.CompareTag("Spike"))
    {
        _gameManager.ResetMap();
    }

    if (other.CompareTag("Portal"))
    {
        _gameManager.ToNextMap();
    }
}
```

Start에서 _gameManager를 찾아오니 _gameManager가 null이 아님을 보장할 필요가 없는 것 아닌가?

→ Start보다 OnTriggerEnter2D가 먼저 호출되는 경우가 있을 수 있음 (Start는 생각보다 매우 늦게 호출됨!)

다른 코드보다 확실히 먼저 호출되게 하고 싶다면 Awake라는 메서드를 쓰면 된다

Github Gist의 PlayerController_0.cs 참조

2. 이번 시간 인트로

이번 시간에 해 볼 것

- 시작 화면과 스테이지 선택 화면 만들기

- 시작 화면, 스테이지 선택 화면은 인게임 화면과 어떻게 다를까?
  - 인게임 화면에서는 게임오브젝트들이 역동적으로 움직였음
  - 시작 화면, 스테이지 선택 화면은 움직이는 게 아니라, 화면상에 고정되어 있어야 함
  - ⇒ 즉, UI가 필요함

  - 인게임 화면에서는 Update에서 방향키를 눌렀는지 매 프레임 확인하고, 이에 따라 플레이어를 이동시킴
  - 시작 화면, 스테이지 화면에서는 버튼이 눌릴 때까지 기다렸다가, 버튼이 눌리면 그제서야 어떤 행동을 취함
  - ⇒ 즉, "비동기"임

3. 시작 화면 UI 만들기

목표

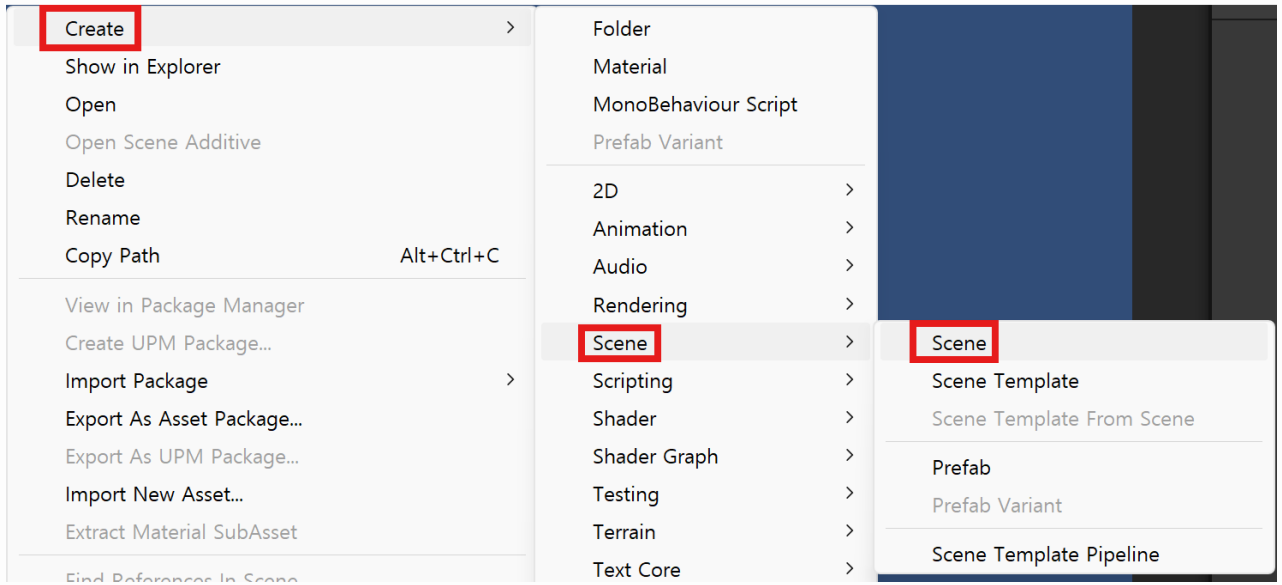
- 시작 화면 만들기
  - 제목 표시용 텍스트
  - 시작 버튼/나가기 버튼



씬 추가하기



Assets/Scenes
폴더로 이동



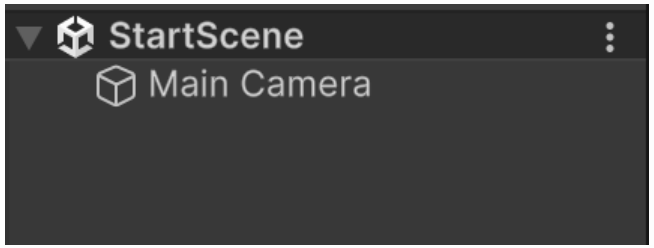
우클릭 후 Create-Scene-Scene 선택



StartScene
생성

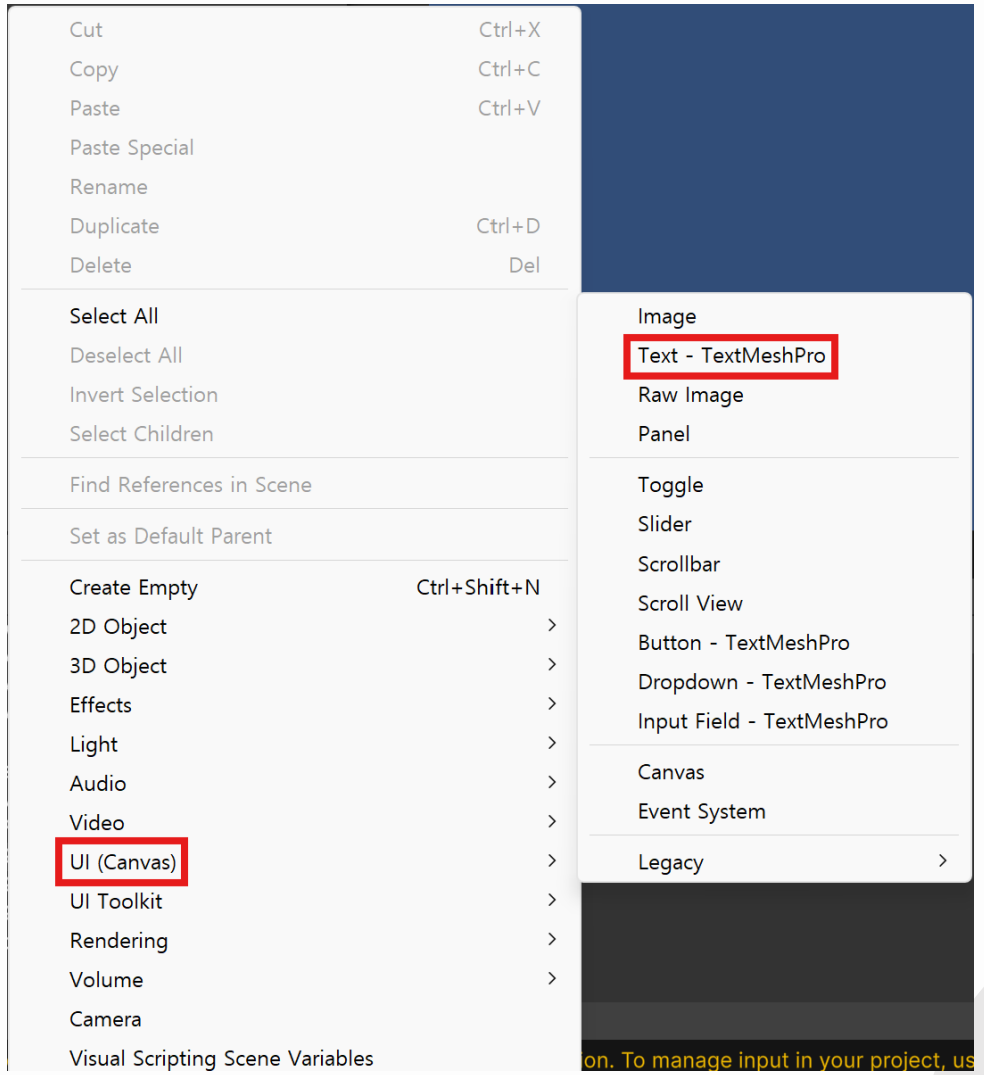
UI 추가하기



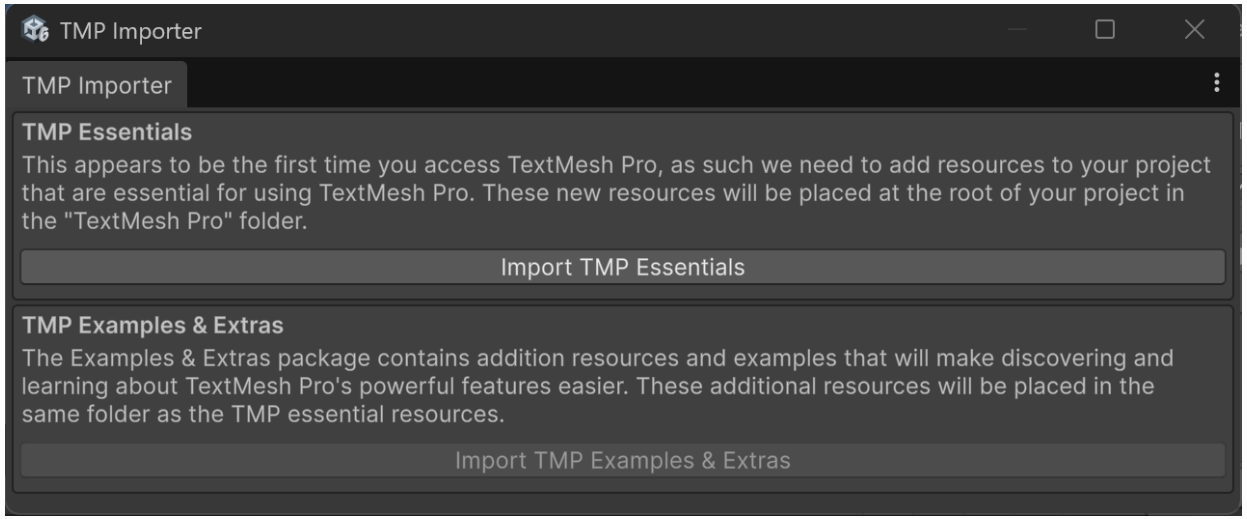


StartScene 더블클릭-StartScene 열기

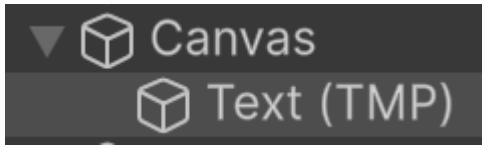
하이어라키 뷰 우클릭-UI(Canvas)-Text 선택



UI 추가하기

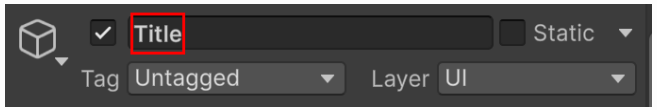


TMP Importer 팝업 창-Import TMP Essentials

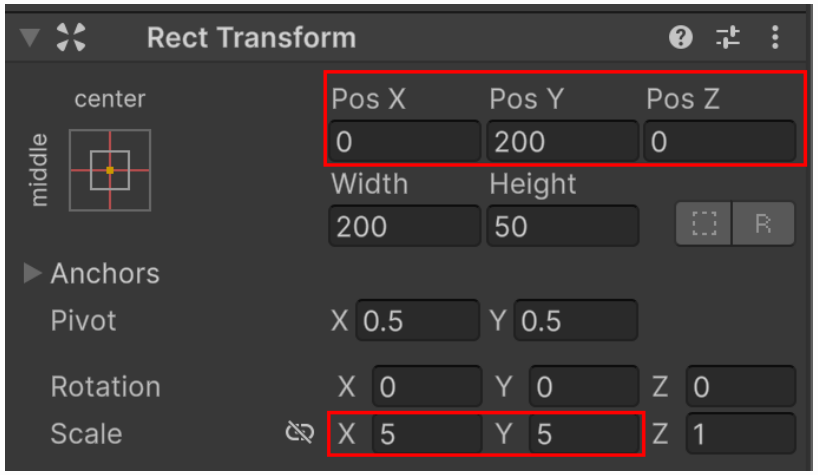


Canvas 아래에 Text (TMP)가 추가됨

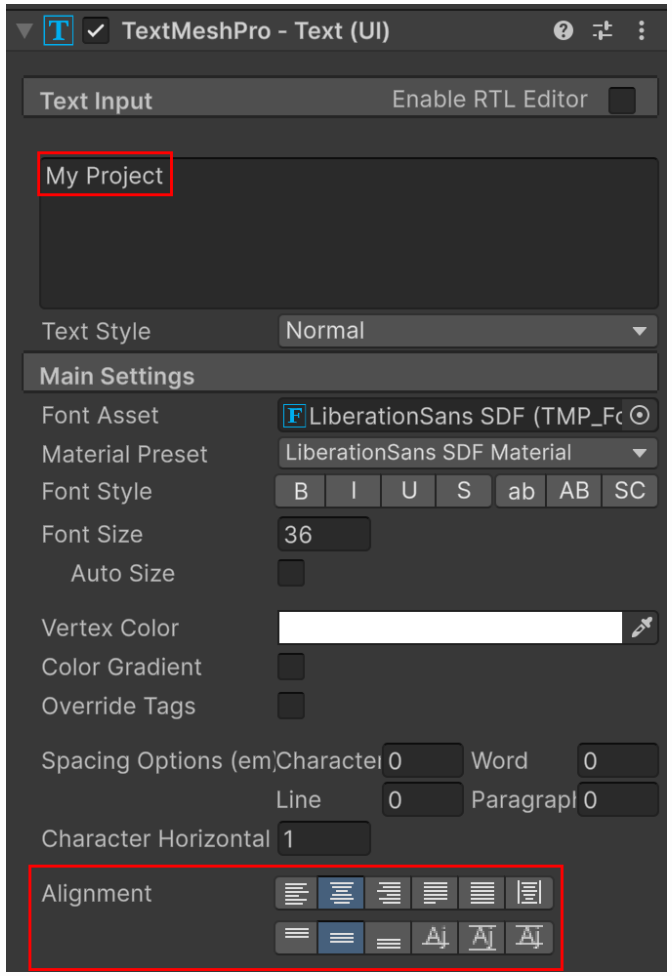
시작 화면 텍스트 추가하기



Text 오브젝트의 이름을
Title로 변경



위치, 크기 설정

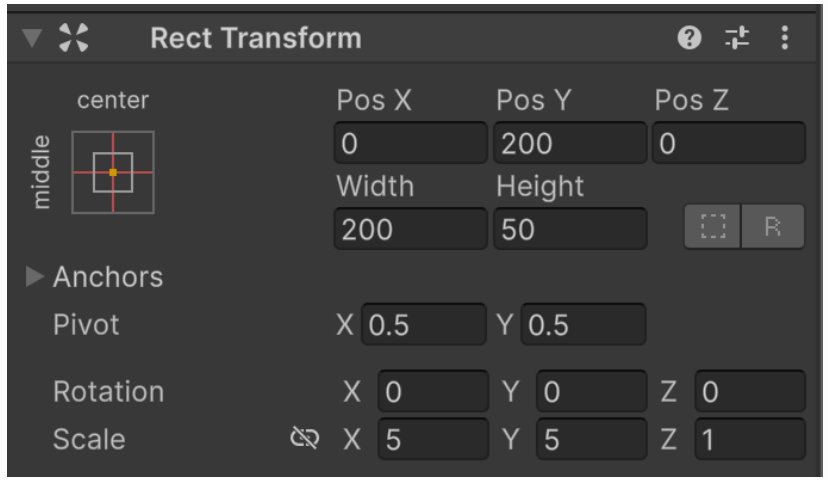


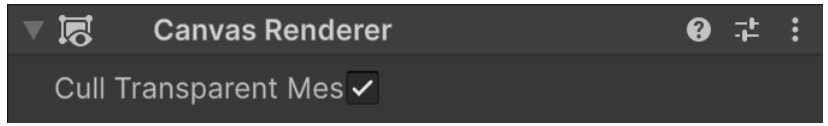
텍스트 내용 수정
정렬 방식을 가로, 세로
모두 가운데 정렬로 변경

Text 게임오브젝트 분석

- 일반적인 게임오브젝트와는 몇 가지 다른 점이 있음
  - Canvas 아래에 생성됨
  - Transform 대신 Rect Transform 존재
  - Sprite Renderer는 없음
  - Canvas Renderer 존재

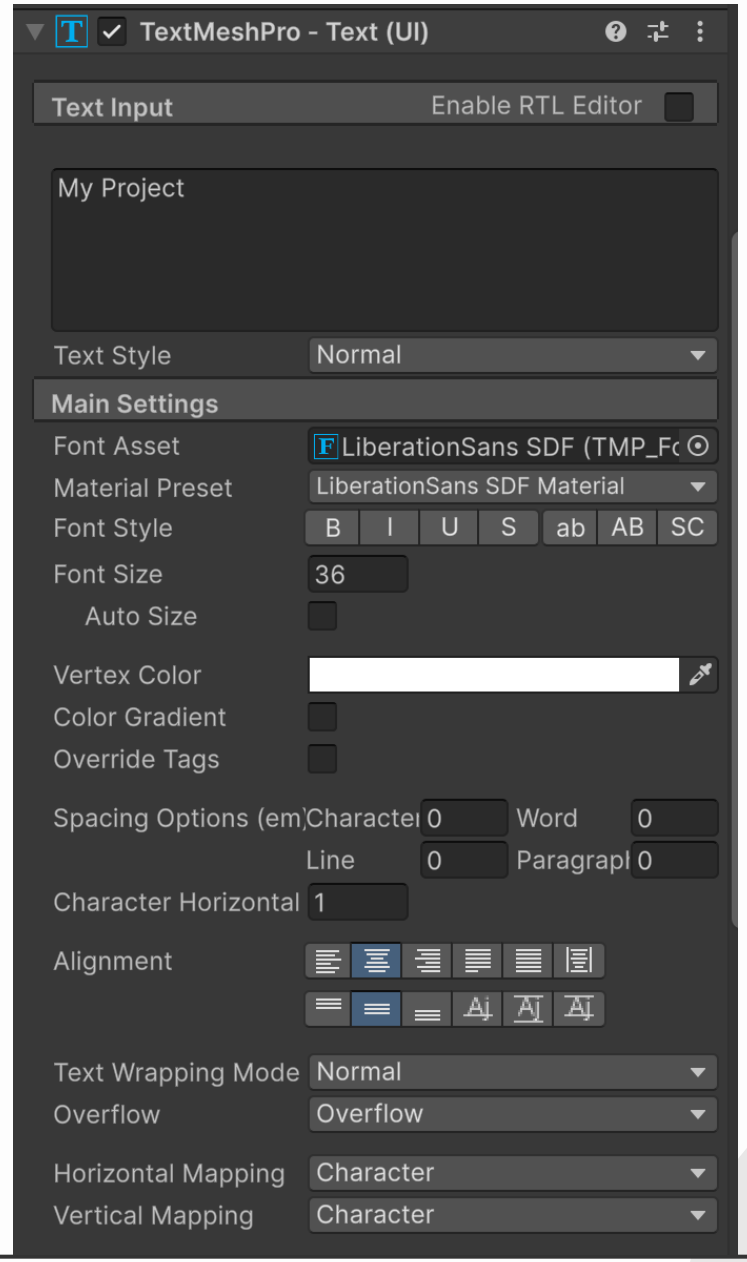
- Rect Transform: 2D UI의 레이아웃을 관리하는 컴포넌트
- Canvas Renderer: 캔버스상의 UI를 렌더링하는 컴포넌트





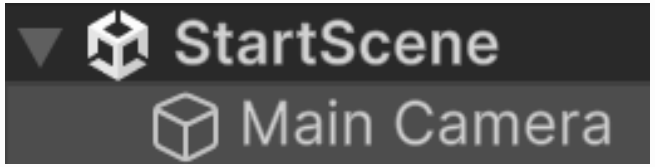
Text 게임오브젝트 분석

- TextMeshPro 컴포넌트
  - 텍스트 내용, 스타일, 폰트, 색, 정렬 등 선택 가능



카메라와 UI

- UI 게임오브젝트는 다른 게임오브젝트와 생성 방식, 컴포넌트 등이 다름
- 이러한 UI의 작동 원리를 이해하기 위해서는 카메라에 대해 알아야 함



- 씬을 처음 생성하면 카메라는 항상 기본적으로 생성되어 있으며, 이 카메라를 삭제하면 게임 뷰에서는 아무것도 보이지 않음

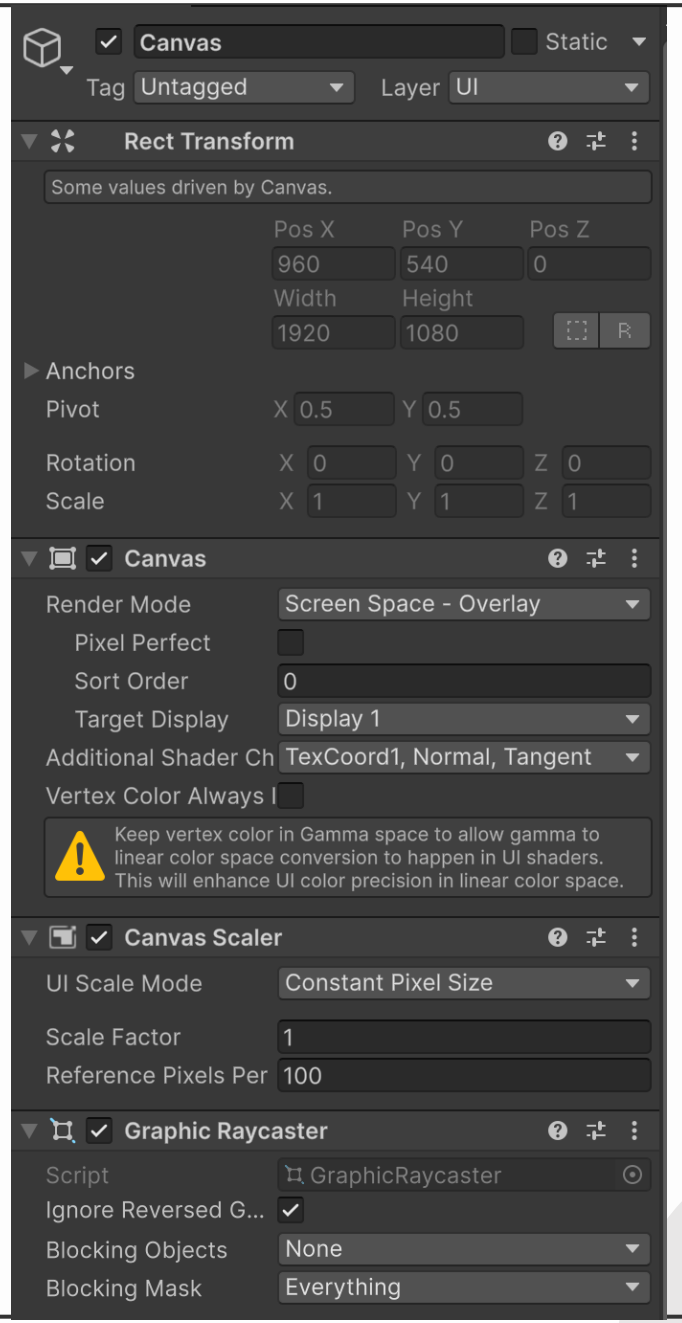
- 카메라가 보는 것이 곧 게임 화면에 나타나는 것 → 카메라가 움직이면 게임오브젝트가 실제로 움직이지 않더라도 화면이 움직이는 것처럼 보임(실제로 FPS 게임 등 카메라를 직접 움직이는 게임은 흔함!)

- 그러나 UI는 카메라의 움직임과 무관하게 항상 화면상에 표시되어야 함 → UI를 일반 게임오브젝트와 다른 방식으로 배치하는 이유

캔버스

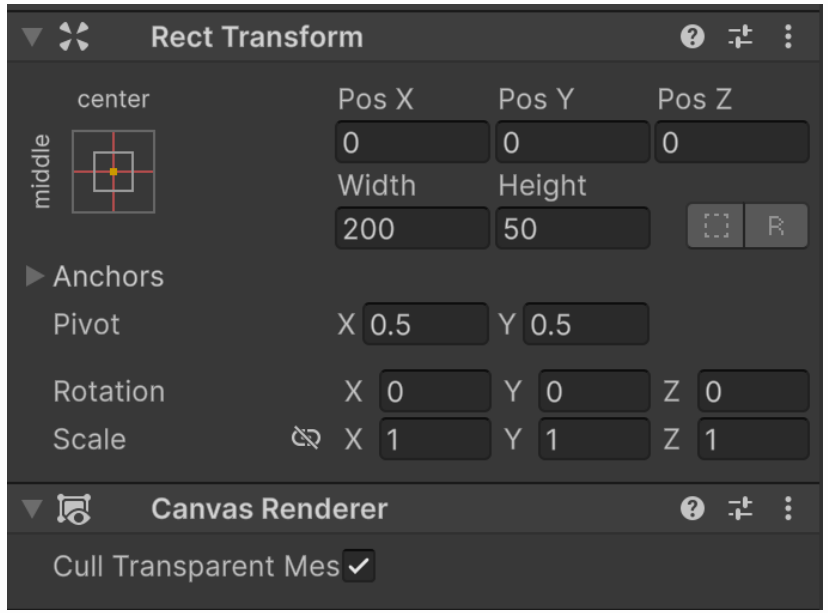
- 모든 UI들이 위치하는 부모 영역
- UI는 캔버스 내에 위치해야만 렌더링됨

- Render Mode: 캔버스가 화면에 렌더링되는 방식 결정
  - Screen Space-Overlay: 화면 위에 UI들이 그려짐
  - Screen Space-Camera: 카메라에서 일정 거리만큼 떨어져서 UI들이 그려짐 → Overlay와는 달리 다른 오브젝트에 의해 가려질 수 있음
  - Screen Space-World: 일반적인 게임오브젝트처럼 씬 안에 직접 배치됨



좌표계

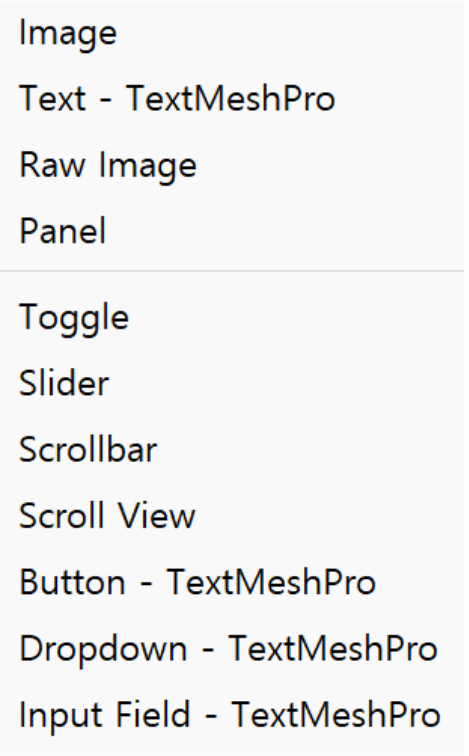
- Rect Transform의 좌표는 Transform의 좌표와 다름
- 일반적인 게임오브젝트는 '월드 좌표계'를 사용하지만, UI 게임오브젝트는 자신이 속한 캔버스를 기준으로 한 '캔버스 로컬 좌표계'를 사용



UI의 종류

- 이미지, 텍스트, 토글, 슬라이더, 스크롤, 버튼, 드롭다운, 입력 필드 등

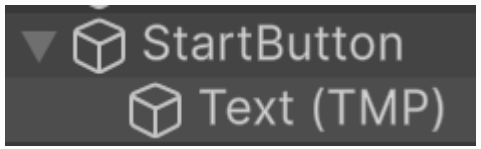
- UI별로 그 UI만의 특성을 나타내기 위한 컴포넌트를 가짐
  - 텍스트는 TextMeshPro 컴포넌트 → 텍스트 내용과 모양 지정
  - 버튼은 Button 컴포넌트 → 클릭 시 상호작용 지정
  - 드롭다운은 Dropdown 컴포넌트 → 선택지 리스트, 선택 시 상호작용 등 지정



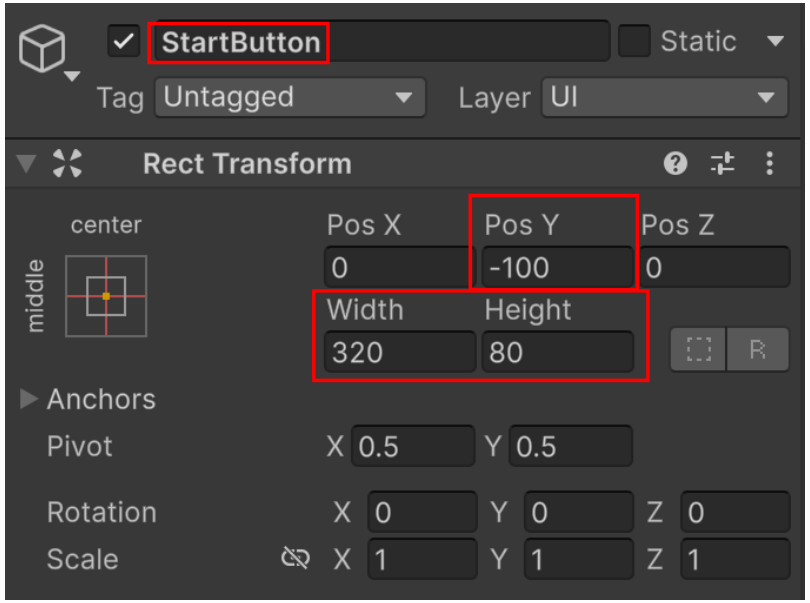
시작 버튼 추가하기



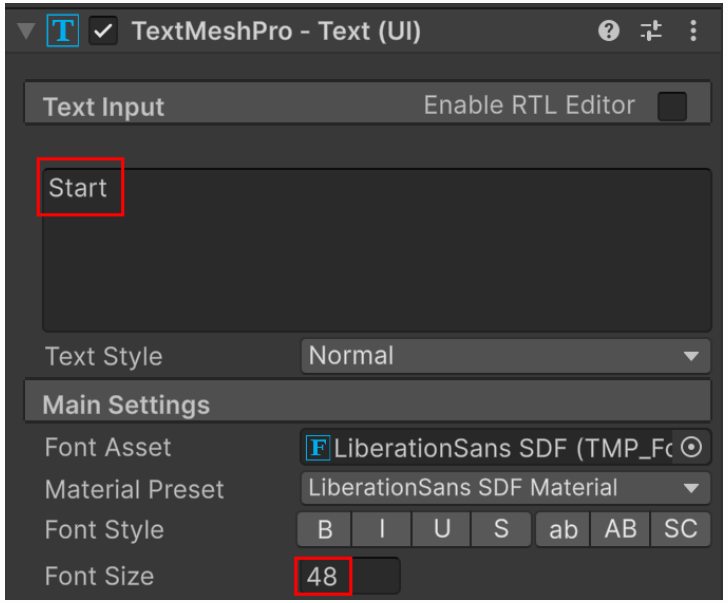
하이어라키 뷰 우클릭-
Create-UI-Button



StartButton에는 자식 오브젝트로 Text가 있음



이름, 위치, 너비/높이 변경



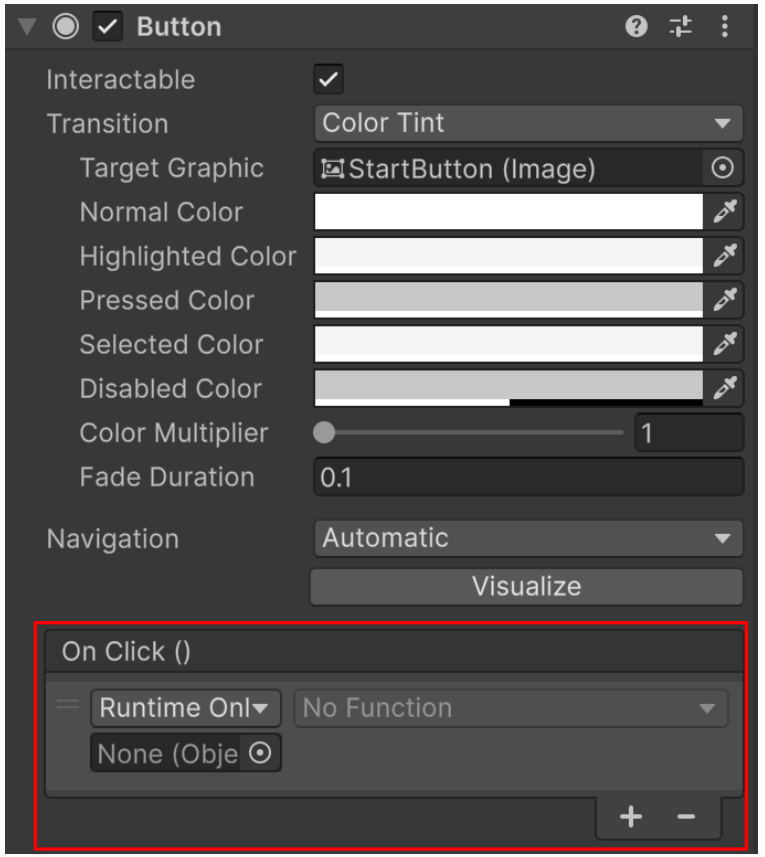
텍스트, 폰트 크기 변경

버튼 클릭 처리하기

- Button 컴포넌트에는 On Click이라는 필드가 있음
  - 버튼이 클릭됐을 때, On Click에 할당된 스크립트의 함수가 호출됨

- On Click에 함수를 할당하는 건 에디터를 통해서도, 스크립트를 통해서도 할 수 있음

- 스크립트를 통한 구현은 다음 장에서 보기로 하고, 여기서는 에디터를 통한 함수 할당을 해 보자



시작 버튼 추가하기



Assets/Scripts에
StartSceneManager
스크립트 생성

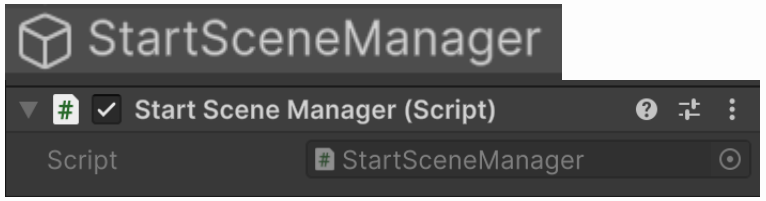
```
using UnityEngine;
using UnityEngine.SceneManagement;

애셋 사용 위치가 없습니다
public class StartSceneManager : MonoBehaviour
{
    public void StartGame()
    {
        SceneManager.LoadScene("SampleScene");
    }

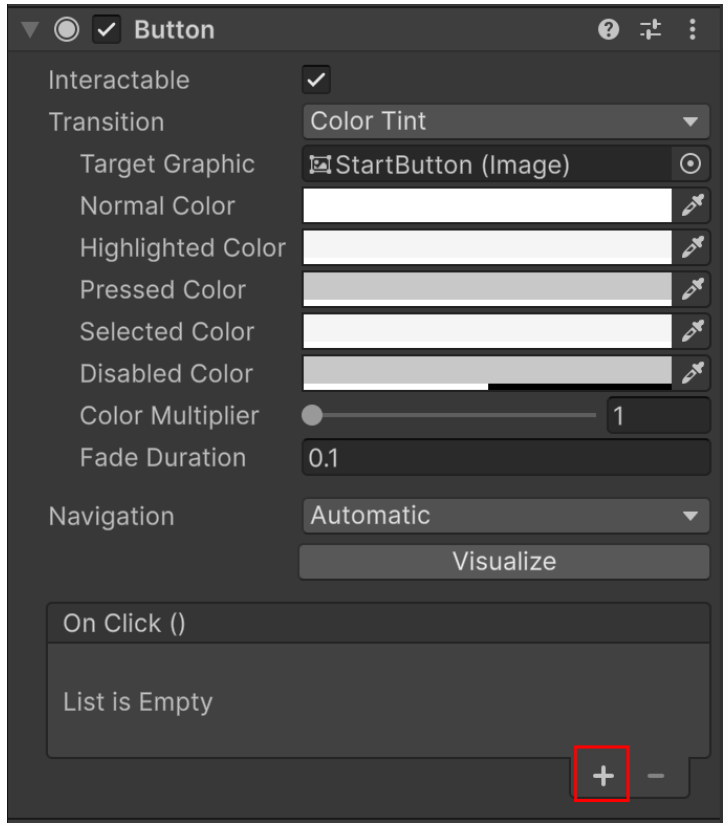
    public void QuitGame()
    {
        Application.Quit();
    }
}
```

코드 작성
코드는 Github Gist의
StartSceneManager_0.cs 참조

시작 버튼 추가하기

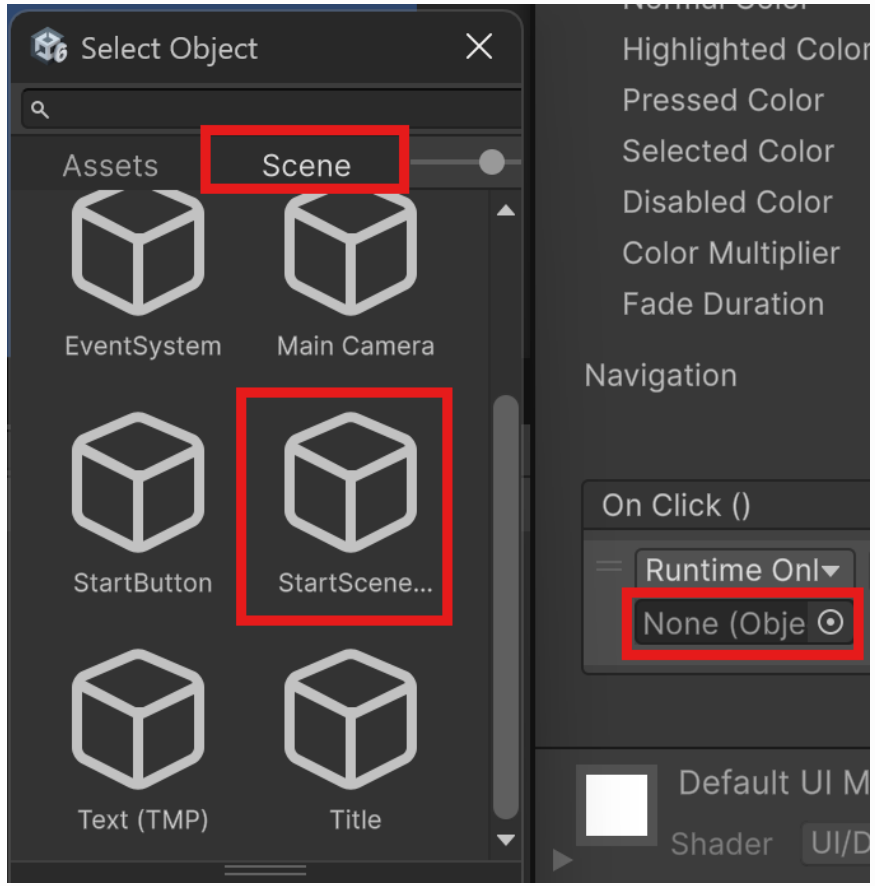


StartSceneManager 게임오브젝트 생성, StartSceneManager 스크립트 적용



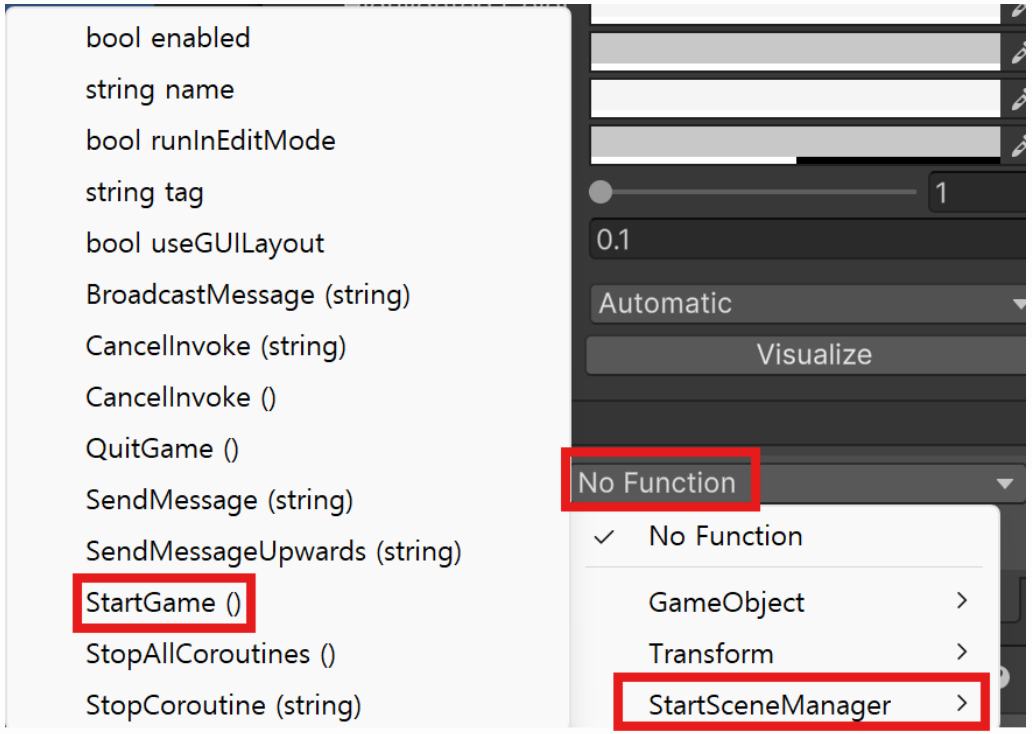
StartButton의 Button 컴포넌트에서 On Click()의 + 클릭

시작 버튼 추가하기



None (Object) 라고 표시된 부분의 ⊙ 클릭
Select Object 팝업 창이 뜨면 Assets 대신 Scene 선택
StartSceneManager 선택

시작 버튼 추가하기

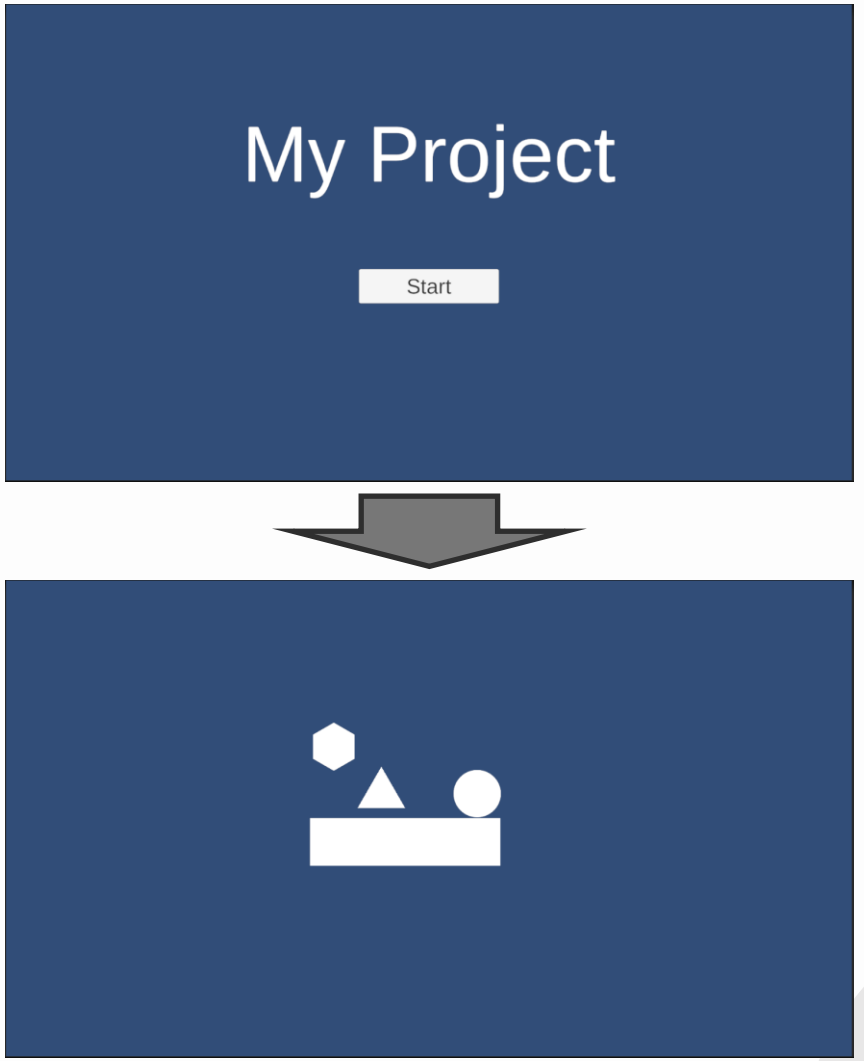


No Function이라고 쓰인 드롭다운 클릭
StartSceneManager 선택
StartGame() 선택

의미:
'이 버튼이 클릭되면'
'StartSceneManager 게임오브젝트에 있는'
'StartSceneManager 컴포넌트의'
'StartGame() 함수를'
호출하라

시작 버튼 추가하기

- 게임 실행 후 시작 버튼을 누르면 실제로 인게임 화면으로 넘어가는 것을 확인할 수 있음



코드 분석

```
using UnityEngine;
using UnityEngine.SceneManagement;

1 애셋 사용 위치
public class StartSceneManager : MonoBehaviour
{
    public void StartGame()
    {
        SceneManager.LoadScene("SampleScene");
    }

    public void QuitGame()
    {
        Application.Quit();
    }
}
```

SceneManager: Unity에서 씬을 관리하는 클래스
LoadScene: 인수로 준 문자열의 이름을 갖는 씬을 불러옴

Application.Quit(): 게임을 종료

실습 2: 종료 버튼 만들기

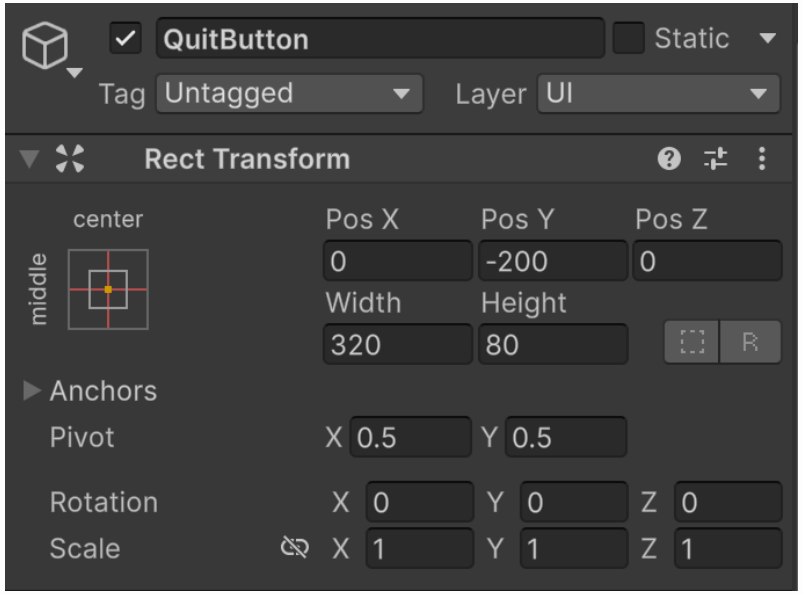
- 눌렸을 때 게임을 종료시키는 종료 버튼도 만들어보자

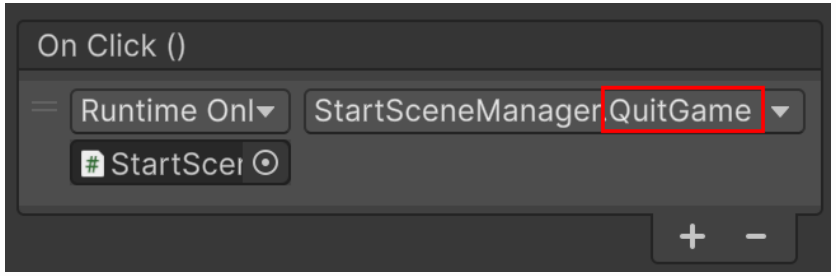
- 힌트
  - 필요한 함수는 이미 있다
  - StartButton을 만들었던 그대로 다시 하면 된다

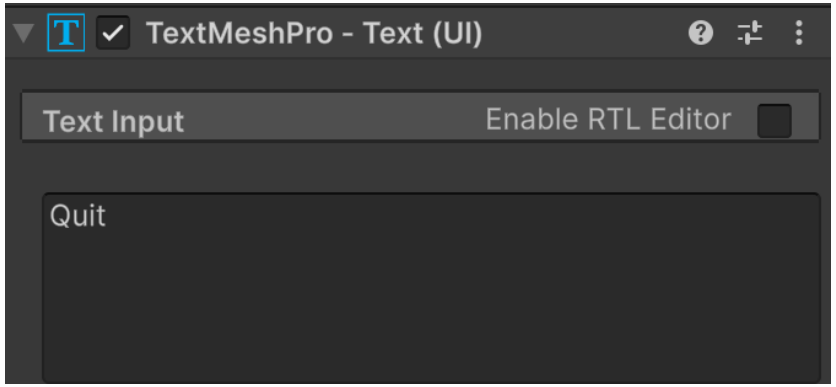
  - 사실 QuitGame은 에디터에서 게임을 종료시키는 게 아니라, 이 게임을 애플리케이션으로 만들어서 실행할 때 애플리케이션을 종료한다는 뜻이다
    - 즉, 지금 단계에서는 작동하지 않는 게 정상이다

실습 2: 종료 버튼 만들기

- 정답







4. 인게임 UI 만들기

그리고 몇 가지 심오한 고민들...

목표

- 스테이지 이름 표시하기
  - 스테이지가 바뀔 때마다 표시되는 내용도 달라짐
- 리셋 버튼 추가하기
  - 누르면 현재 스테이지를 다시 시작

- '진짜 프로그래머가 할 법한 고민' 해 보기

스테이지 이름 표시하기

- 매 스테이지마다, 그 스테이지의 이름을 표시함
- 필요한 기능

스테이지별로 이름 데이터 저장

이름 표시 UI

스테이지가 바뀔 때 이름도 바뀜

- 이 중 지금 할 수 있는 건?

실습 3: 지난 시간 복습

- 맵 데이터에 맵 이름도 저장할 수 있게 해 보자

- 힌트
  - 맵 데이터는 무엇으로 저장하고, 어디서 정의하더라?
  - 말 그대로 지난 시간 복습이다

실습 3: 지난 시간 복습

- 정답: MapData.cs

```
using System;
using System.Collections.Generic;
using UnityEngine;

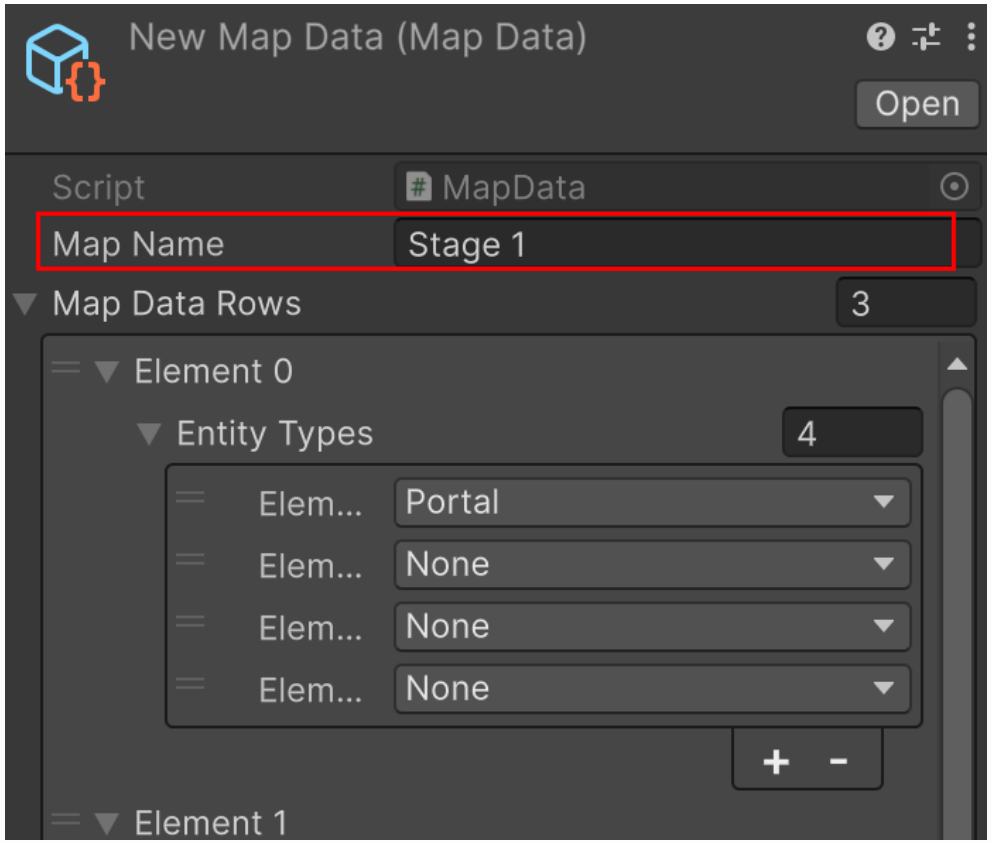
[CreateAssetMenu(menuName = "Scripts/Map Data")]
1 애셋 사용 위치  2 사용 위치
public class MapData : ScriptableObject
{
    [Serializable]
    2 사용 위치
    public class MapDataRow
    {
        public List<GameManager.EntityType> entityTypes = new List<GameManager.EntityType>();
    }

    public string mapName;  변경되지 않음
    public List<MapDataRow> mapDataRows = new List<MapDataRow>();  직렬화 가능
}
```

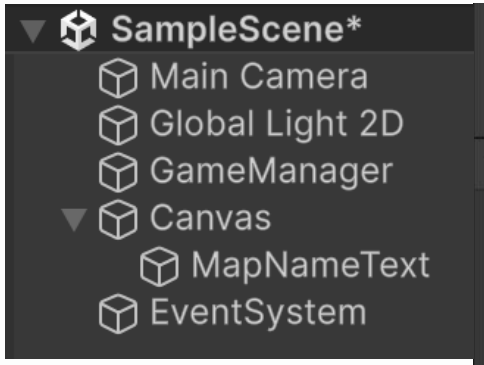
코드는 Github Gist의 MapData_0.cs 참조

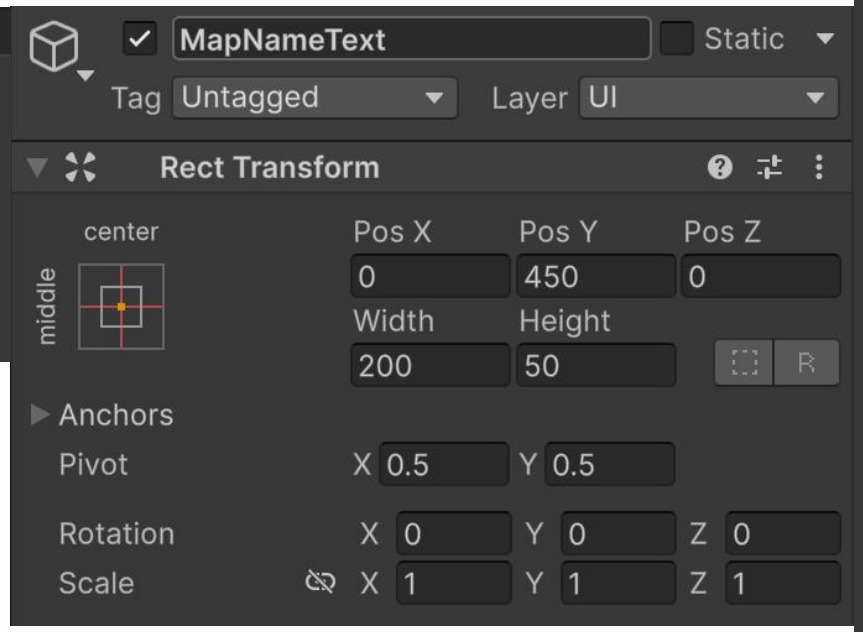
스테이지 이름 표시하기

- 지난 시간에 생성했던 MapData ScriptableObject에서 Map Name 변경 가능

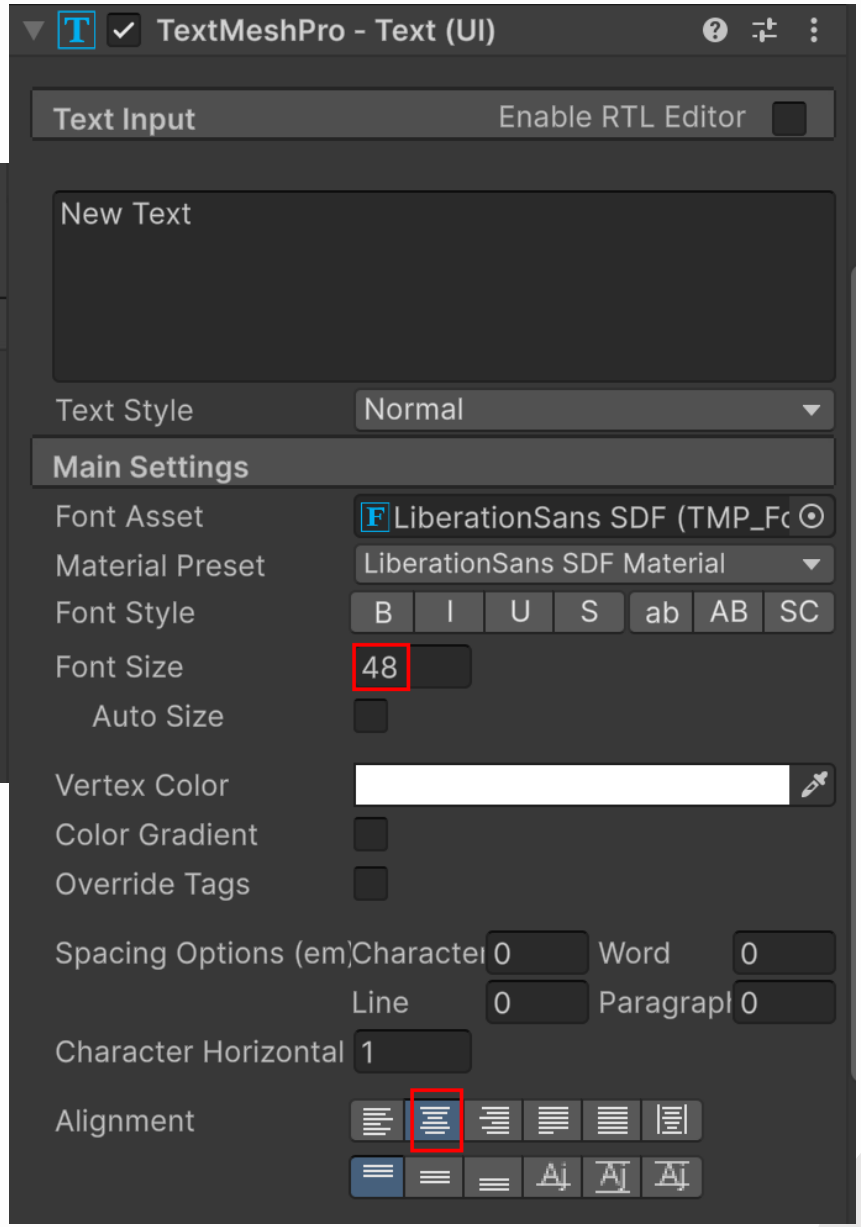


스테이지 이름 표시하기





SampleScene으로 이동 후
MapNameText 생성



스테이지 이름 표시하기

- 나머지는 다 됐는데, '스테이지가 바뀔 때마다 스테이지 이름을 바꾸는' 부분은 어떻게 구현해야 할까?
- 한 가지 방법: GameManager에서 SerializeField로 텍스트 게임오브젝트를 받아서, 스테이지를 바꿀 때마다 텍스트도 바꾸면 되지 않을까?
- 이미 패턴을 파악했다면 눈치챘겠지만, 이런 형식적인 질문과 형식적인 '한 가지 방법'이 늘 그렇듯 좋은 방법은 아님

왜 좋은 방법이 아닐까?

- 한 가지 방법: GameManager에서 SerializeField로 텍스트 게임오브젝트를 받아서, 스테이지를 바꿀 때마다 텍스트도 바꾸면 되지 않을까?

- 문제점
  - 스테이지를 바꿀 때마다 텍스트가 바뀌어야 한다! → 스테이지 정보는 GameManager에 있으니 텍스트 교체도 GameManager가 하게 하자
  - 리셋 버튼을 누르면 스테이지가 리셋돼야 한다! → 오브젝트 생성은 GameManager가 하니 버튼 클릭 감지도 GameManager가 하게 하자
  - 스테이지 시작할 때 대사를 넣고 싶다! → 스테이지 시작은 GameManager가 하니 대사 띄우기랑 다음 대사 넘기기까지 GameManager가 하게 하자

왜 좋은 방법이 아닐까? - 절망편



왜 좋은 방법이 아닐까? - 절망편 2(희망 따윈 없다)



왜 좋은 방법이 아닐까?

- 문제점: GameManager가 모든 자잘한 역할을 다 담당하게 하면 GameManager의 역할이 너무 커짐
- 역할이 너무 커졌을 때의 문제점
  - GameManager의 정체성이 모호해짐
  - 코드 사이의 의존성이 높아짐(전부 한 스크립트 안에 있으므로)
  - 쉽게 말해, 코드가 꼬이기 쉬워짐

- 이거 하나만 GameManager한테 맡기고 다른 건 다른 방식으로 해결할테니 한 번만 봐주세요 ㅠㅠ
  - 일단 한 번 이렇게 코드를 짜고 나면, 다음에도 똑같이 짜고 싶어질 것임
  - 처음부터 코드를 제대로 짜는 습관을 들이자

매우 심오한 고민

- GameManager 안에 코드를 다 집어넣는 게 좋은 방법이 아니라면, 어떻게 하는 게 좋은 방법인가?

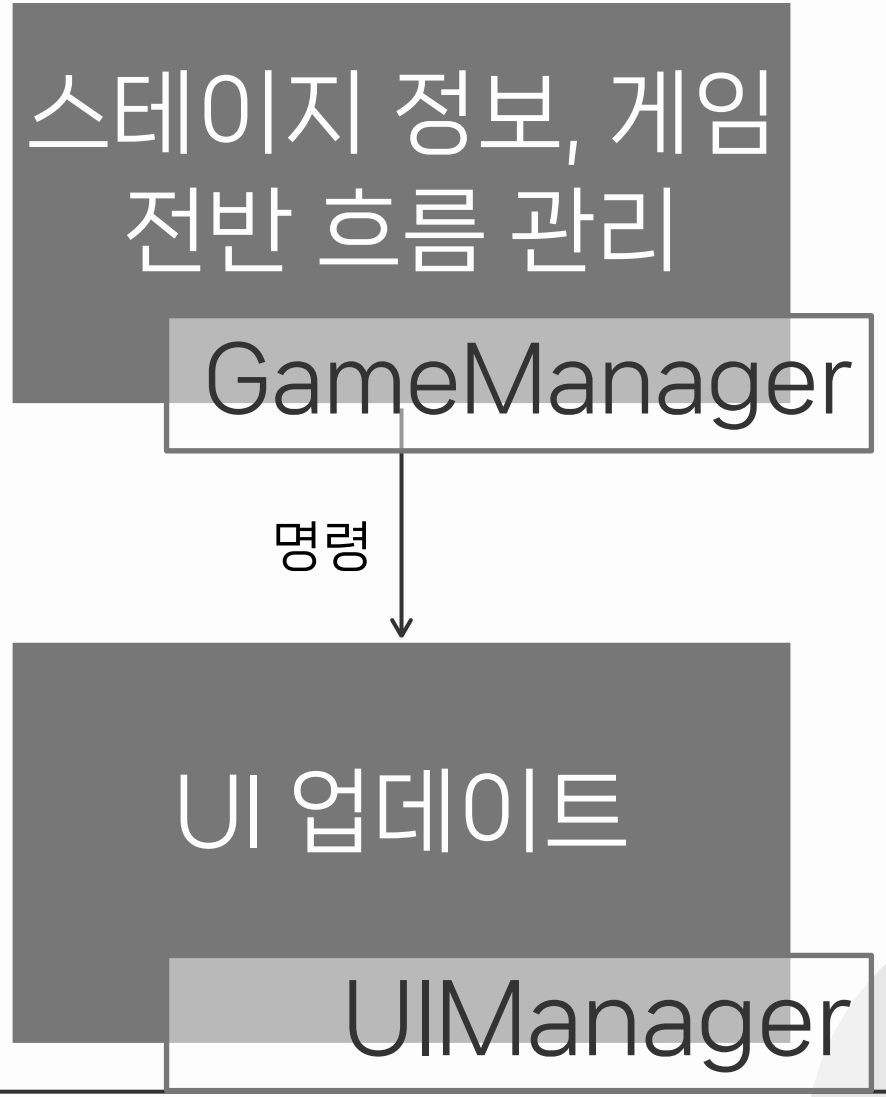
- 한 가지 답: UI 업데이트를 담당하는 스크립트를 따로 만들자!
- '한 가지 답'인 이유: 틀려서가 아니라, 정말로 많은 방법 중 하나에 불과하기 때문
  - 게임 개발에 정답은 없다!
  - ~~확실한 오답이 있을 뿐...~~

매우 심오한 고민

- 우리의 계획

- GameManager는 스테이지 정보와 게임 흐름(시작/다음 스테이지/리셋 등)만 관리
- UIManager에서 UI의 내용 갱신 담당

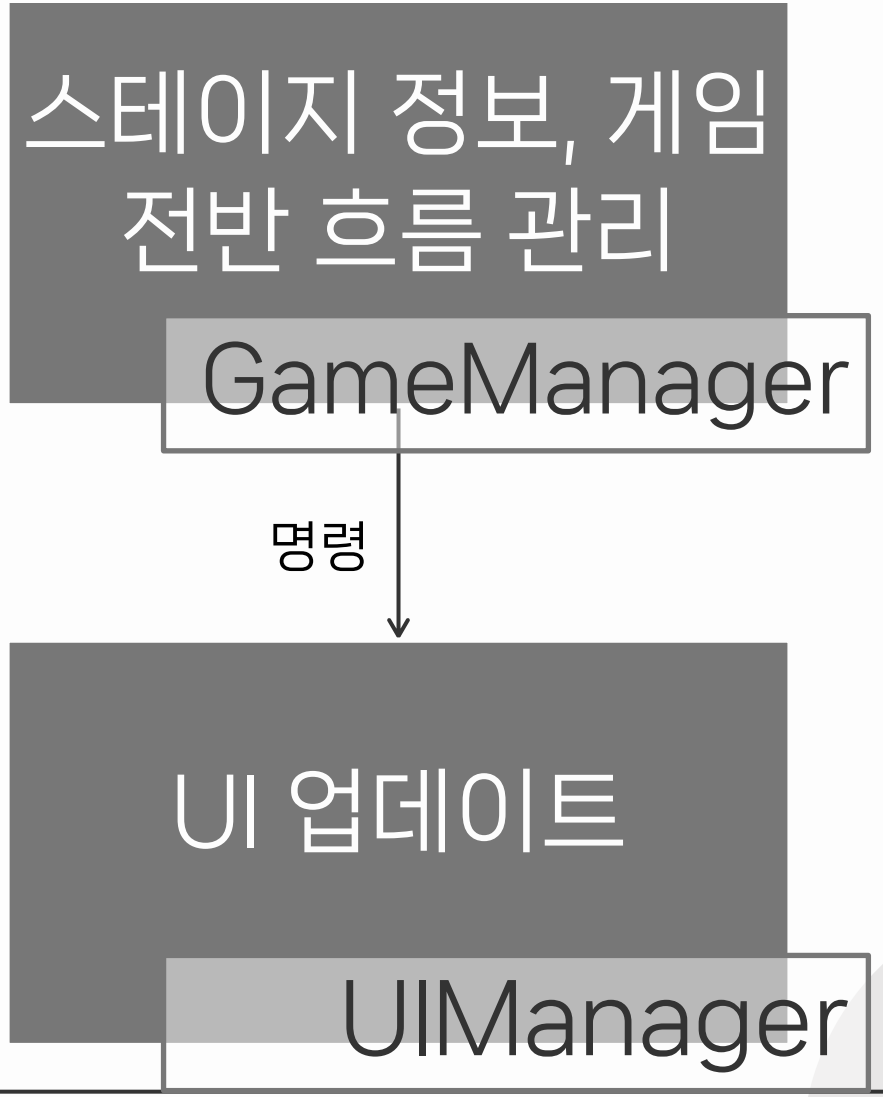
- 질문: GameManager에서 UIManager에 어떻게 UI를 갱신하라는 명령을 내릴 것인가?



매우 심오한 고민

- GameManager에서 UIManager에 어떻게 UI를 갱신하라는 명령을 내릴 것인가?

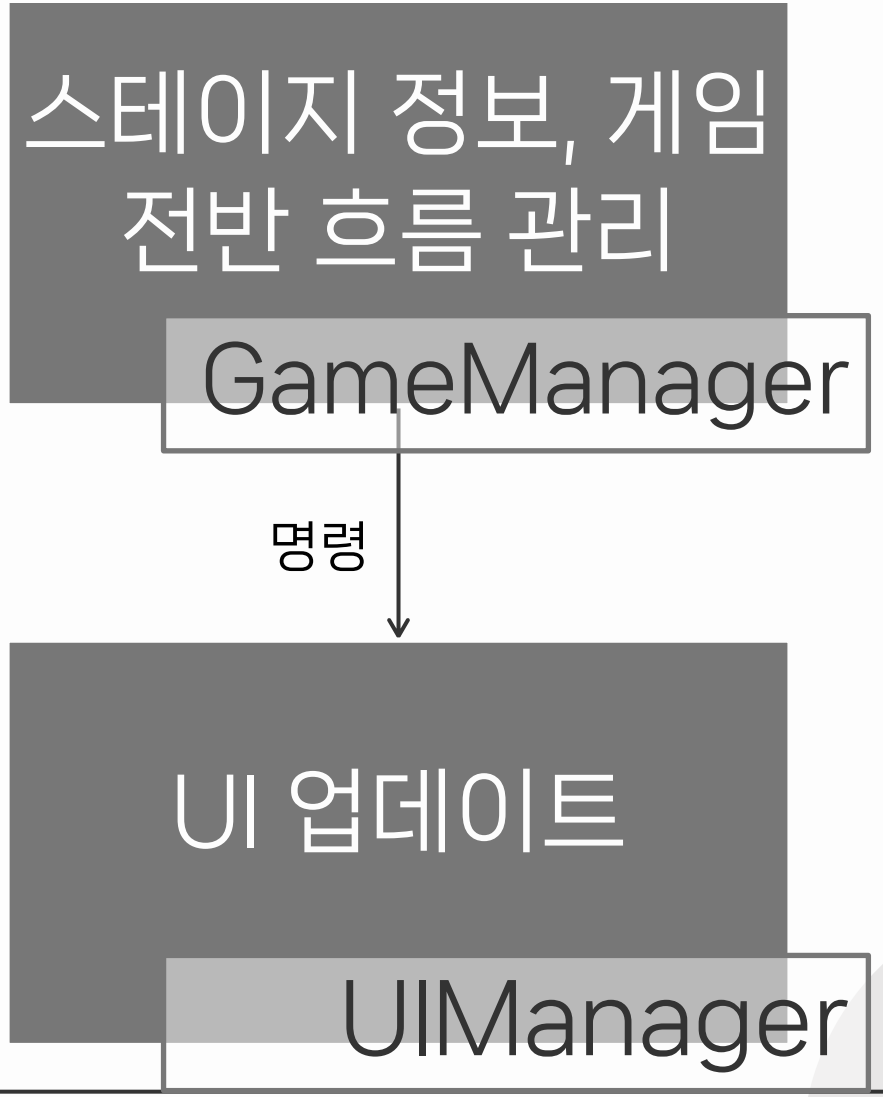
- 스테이지 클리어/게임 오버를 구현했던 것처럼 GameManager가 UIManager의 메서드를 호출하게 할까?
  - 그렇게 하면 GameManager가 모든 걸 다 하는 것과 다를 게 없음
  - 오히려 GameManager가 UIManager의 정보를 알고 매번 UIManager.(...) 꼴로 호출해야 하므로 더 나쁠 수도 있음



매우 심오한 고민

- 객체 지향 프로그래밍의 원칙 중 하나: 의존성 역전 원칙(Dependency Inversion Principle, DIP)
  - 구체적인 클래스가 추상적인 클래스에 의존해야지, 그 반대가 되어서는 안 됨
- 즉, GameManager가 UIManager에 의존해서는 안 됨

- 어떻게 하면 DIP를 해치지 않으면서 GameManager가 UIManager를 호출하게 할 수 있을까?

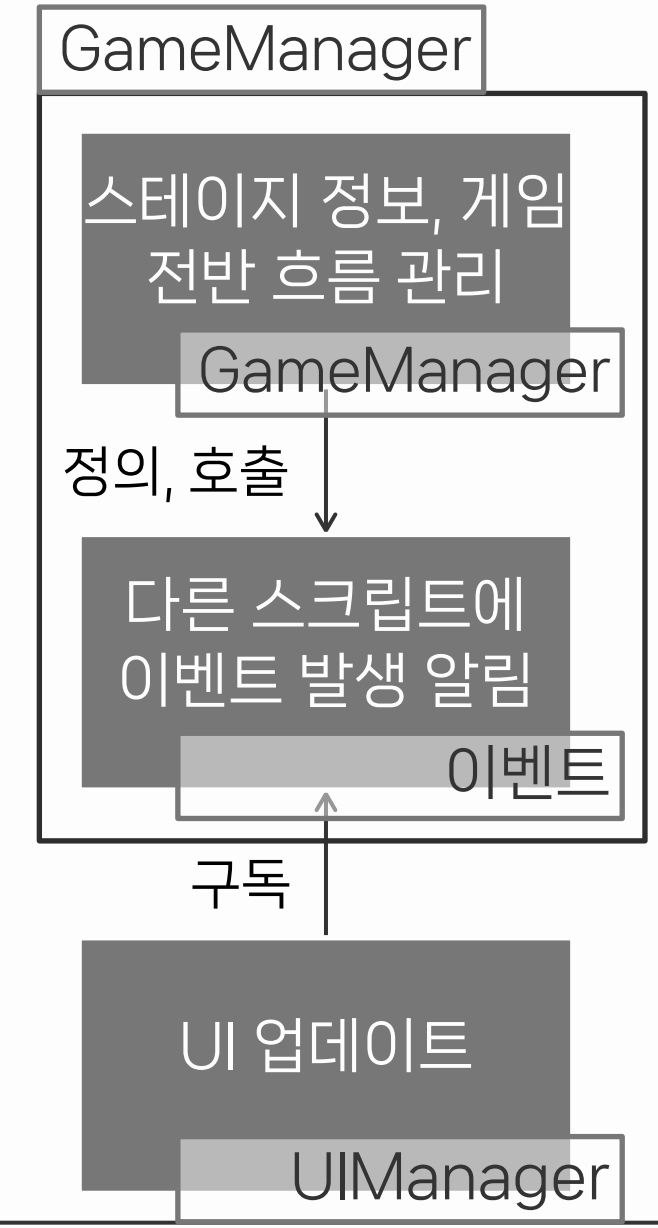


옵저버 패턴

- 이런 문제는 게임(또는 소프트웨어) 개발을 하는 사람이라면 누구나 겪을 만한 문제이기에, 문제를 해결하는 표준적인 방법이 있음
- 이를 '디자인 패턴'이라고 함

- 이 경우에 쓸 수 있는 디자인 패턴: 옵저버 패턴

- GameManager에 이벤트를 정의하고, 특정 조건에서 이 이벤트를 호출
- 그 이벤트를 구독(subscribe)하고 있는 모든 스크립트에 알림이 감 → 스크립트별로 등록해 둔 함수 실행



한 가지 더

- 지금까지는 GameManager를 호출하기 위해서 미리 **GameObject.Find**로 GameManager를 찾고, **GetComponent**로 GameManager 스크립트에 접근했음
- 그러나 이는 별로 좋은 방법이 아님
  - 문제 1: **Find**, **GetComponent** 모두 비싼 연산이므로 성능 문제 발생
  - 문제 2: 이름으로 게임오브젝트를 찾으므로, GameManager의 이름이 바뀌면 오브젝트를 찾을 수 없음
  - 문제 3: GameManager를 호출해야 하는 스크립트마다 위의 방법을 써서 GameManager를 미리 찾아 두어야 하는데, 이 부분이 중복됨 → DRY 위배!

싱글톤 패턴

- 해법: 싱글톤 패턴
- 클래스에 static으로 그 클래스의 인스턴스를 저장하도록 함
- 인스턴스는 하나만 생성되도록 보장
- 다른 스크립트에서 해당 클래스의 인스턴스 접근 시 **(클래스 이름)**.Instance.(...)와 같이 쓰면 됨

싱글톤 패턴: 코드 분석

- GameManager.cs에 다음과 같이 작성(코드는 Github Gist의 GameManager_4.cs 참조)

- public static GameManager Instance: GameManager 클래스에서 공통으로 들고 있는 Instance라는 객체를 선언
- private void Awake: 게임 중 이 스크립트가 처음 생성됐을 때 가장 먼저 실행됨(Start보다도 먼저)
- Instance가 null이면 아직 인스턴스가 없다는 뜻이므로 자기 자신을 인스턴스로 설정
- Instance가 null이 아니면 이미 인스턴스가 있다는 뜻이므로 오류를 발생시키거나, 이 게임오브젝트를 파괴

```
public static GameManager Instance;

이벤트 함수
private void Awake()
{
    if (Instance == null)
    {
        Instance = this;
    }
    else
    {
        Destroy(gameObject);
    }
}
```

실습 4: 싱글톤 패턴으로 바꾸기

- PlayerController의 코드를 싱글톤 패턴으로 바뀐 GameManager에 맞춰 바꿔보자

- 힌트
  - GameManager의 Instance는 static이다
    - 기억나지 않는다면 3주차 9~10페이지를 보자
  - Instance를 호출하면 되므로, _gameManager를 찾아올 필요가 없어진다

실습 4: 싱글톤 패턴으로 바꾸기

- 정답

```
void Start()
{
    _rb = GetComponent<Rigidbody2D>();
    _gameManager = GameObject.Find("GameManager").GetComponent<GameManager>();
}

private float _speed = 5.0f;
[SerializeField] private float _jumpForce = 500.0f;   변경되지 않음
private Rigidbody2D _rb;
private bool _isJumping = false;
private GameManager _gameManager;
```

```
void OnTriggerEnter2D(Collider2D other)
{
    if (_gameManager == null)
    {
        return;
    }

    if (other.CompareTag("Spike"))
    {
        GameManager.Instance.ResetMap();
    }

    if (other.CompareTag("Portal"))
    {
        GameManager.Instance.ToNextMap();
    }
}
```

코드는 Github Gist의 PlayerController_1.cs 참조

다시 돌아와서...

- 이벤트는 어떻게 선언하고, 호출하고, 구독하는가?
- 이벤트 선언: **public event Action<T> (이벤트 이름)**
  - T 타입을 인수로 받는 Action 선언(타입은 여러 개 쓸 수도 있음)
  - Action: delegate의 일종으로, 반환값이 없는 함수에 대한 delegate
  - delegate: 함수에 대한 참조 → 함수를 변수처럼 저장하는 기능이라고 생각하면 됨
  - event: delegate의 호출 방식을 제한 → +=, -=으로만 함수를 할당할 수 있으며, 외부 클래스에서는 Invoke할 수 없음
- 이벤트 구독: **(이벤트 이름) += (메서드 이름)**
  - 구독 해제: **(이벤트 이름) -= (메서드 이름)**
  - T에 명시된 타입과 같은 타입의 인수를 받는 메서드만 구독할 수 있음
- 이벤트 호출: **(이벤트 이름)?.Invoke((인수))**
  - ?.: 그냥 .으로 호출할 경우 이벤트가 null일 수 있음 → ?.는 앞에 오는 객체가 null이면 뒤에 오는 메서드나 속성을 평가하지 않고 null 반환
  - 이벤트가 null이 되는 경우: 구독하고 있는 메서드가 없을 때
  - 이벤트를 Invoke하면 그 이벤트를 구독한 모든 메서드가 실행됨

옵저버 패턴 구현하기 – GameManager

```
public event Action<string> OnMapLoaded;
private List<GameObject> _spawnedObjects = new List<GameObject>();

3 사용 위치
private void LoadMap(int idx)
{
    int height = _mapDatas[idx].mapDataRows.Count;
    for (int i = 0; i < height; i++)
    {
        int width = _mapDatas[idx].mapDataRows[i].entityTypes.Count;
        for (int j = 0; j < width; j++)
        {
            Vector3 coord = IndexToCoord( rowIdx: i,  colIdx: j, width, height);
            EntityType entityType = _mapDatas[idx].mapDataRows[i].entityTypes[j];
            if (entityType != EntityType.None)
            {
                _spawnedObjects.Add(Instantiate(_entityDict[entityType], coord, Quaternion.identity));
            }
        }
    }
    OnMapLoaded?.Invoke(_mapDatas[idx].mapName);
}
```

코드는 Github Gist의 GameManager_5.cs 참조

코드 분석

```
public event Action<string> OnMapLoaded;
private List<GameObject> _spawnedObjects = new List<GameObject>();

3 사용 위치
private void LoadMap(int idx)
{
    int height = _mapDatas[idx].mapDataRows.Count;
    for (int i = 0; i < height; i++)
    {
        int width = _mapDatas[idx].mapDataRows[i].entityTypes.Count;
        for (int j = 0; j < width; j++)
        {
            Vector3 coord = IndexToCoord( rowIdx: i,  colIdx: j, width, height);
            EntityType entityType = _mapDatas[idx].mapDataRows[i].entityTypes[j];
            if (entityType != EntityType.None)
            {
                _spawnedObjects.Add(Instantiate(_entityDict[entityType], coord, Quaternion.identity));
            }
        }
    }
    OnMapLoaded?.Invoke(_mapDatas[idx].mapName);
}
```

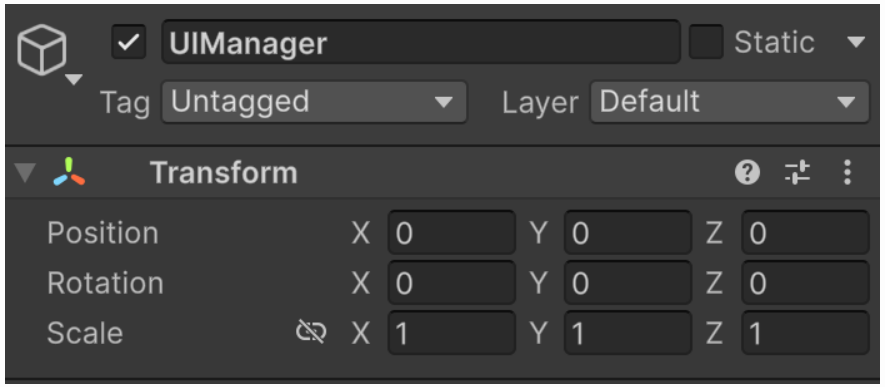
OnMapLoaded 이벤트 선언

LoadMap에서 OnMapLoaded 호출
인수로는 맵 이름 전달

옵저버 패턴 구현하기 - UIManager



UIManager
스크립트 생성



UIManager 게임오브젝트
생성(empty GameObject)

UIManager.cs 작성

코드는 Github Gist의 UIManager_0.cs 참조

```
using UnityEngine;

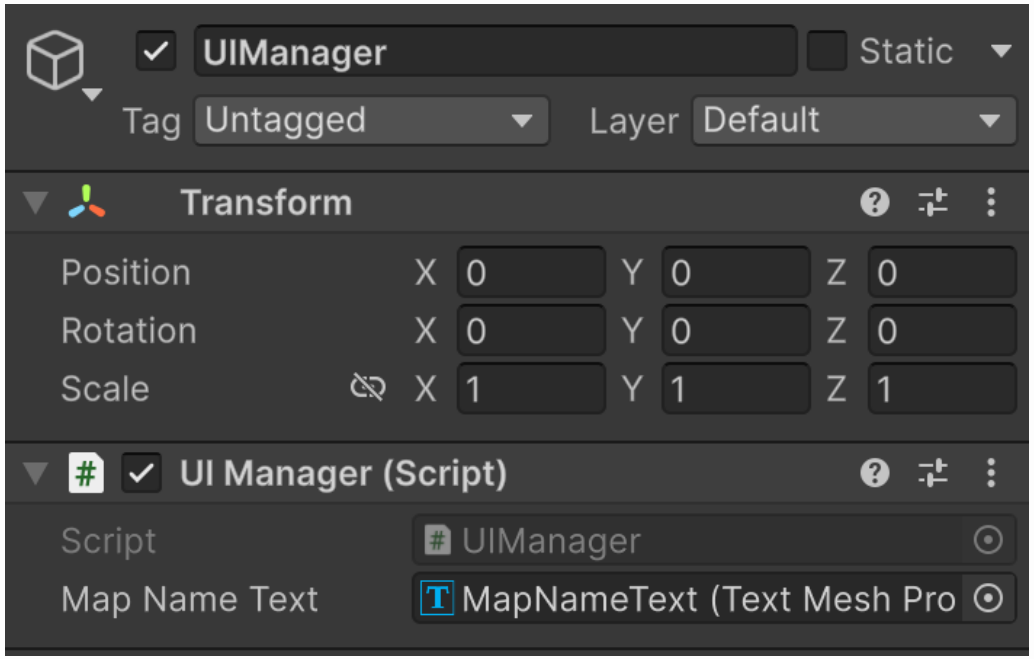
1 애셋 사용 위치
public class UIManager : MonoBehaviour
{
    [SerializeField] private TMPro.TextMeshProUGUI _mapNameText;

    이벤트 함수
    void Awake()
    {
        GameManager.Instance.OnMapLoaded += OnMapLoaded;
    }

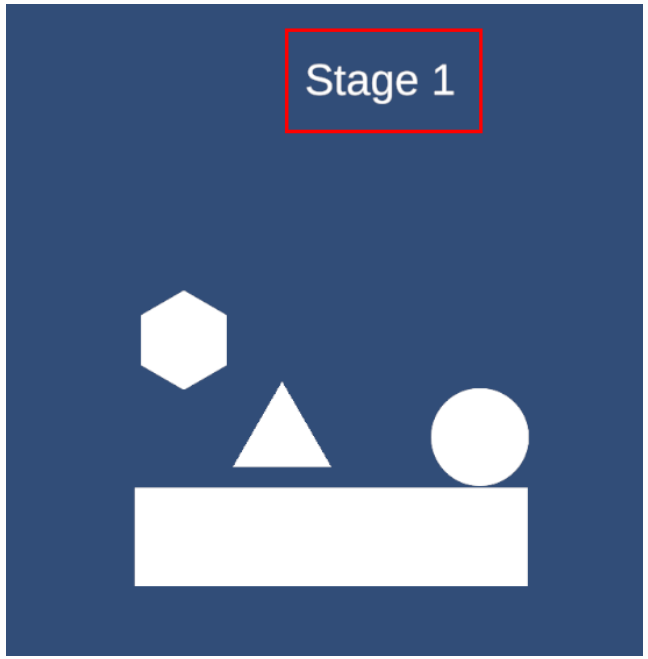
    이벤트 함수
    void OnDestroy()
    {
        GameManager.Instance.OnMapLoaded -= OnMapLoaded;
    }

    2 사용 위치
    void OnMapLoaded(string mapName)
    {
        _mapNameText.text = mapName;
    }
}
```

옵저버 패턴 구현하기



UIManager에 스크립트 적용
Map Name Text에는 MapNameText 게임오브젝트의 TextMeshPro 컴포넌트 지정



실행하면 실제로 앞서 지정한 맵 이름이 출력되는 것을 볼 수 있음

코드 분석

```
using UnityEngine;

☢ 1 애셋 사용 위치
public class UIManager : MonoBehaviour
{
    [SerializeField] private TMPro.TextMeshProUGUI _mapNameText;

    ☢ 이벤트 함수
    void Awake()
    {
        GameManager.Instance.OnMapLoaded += OnMapLoaded;
    }

    ☢ 이벤트 함수
    void OnDestroy()
    {
        GameManager.Instance.OnMapLoaded -= OnMapLoaded;
    }

    ↗ 2 사용 위치
    void OnMapLoaded(string mapName)
    {
        _mapNameText.text = mapName;
    }
}
```

MapNameText의 TMPro 컴포넌트를 SerializeField로 받음

GameManager.Instance의 OnMapLoaded 이벤트 구독

객체에서 이벤트를 구독했다면 반드시 객체가 파괴될 때 이벤트 구독을 해제해야 함(그렇지 않으면 메모리 누수 발생)

OnMapLoaded 이벤트 발생 시 MapNameText의 텍스트를 인수로 전달한 mapName으로 바꿈

디자인 패턴에 관하여...

- 여기에서 소개된 디자인 패턴 외에도 다양한 디자인 패턴이 있음
  - 이벤트 버스 패턴, 방문자 패턴, 오브젝트 풀 패턴 등
- 모든 디자인 패턴에는 장단점이 있음
  - 대표적인 예: 싱글톤 패턴 - 구현이 간단하지만, 남발할 경우 코드 간 의존성이 강해짐
- 모든 상황에 보편적으로 적용되는 최선의 디자인 패턴은 없음 → 각각의 경우에 어떤 디자인 패턴이 최적일지 스스로 판단하여 적용하자

5. 인게임 UI 만들기 2

다음으로 할 일

- 리셋 버튼 추가하기

- 마찬가지로 심오한 고민: 리셋 버튼은 어떻게 처리해야 할까?
  - 맵 제목과의 차이점: 맵 제목은 호출 흐름이 GameManager → UIManager였는데, 리셋 버튼은 버튼 눌림 이벤트 감지 → GameManager의 ResetGame 함수 호출 형태로 이루어짐

- 오히려 더 쉽다!
  - 더 구체적인 클래스가 더 추상적인 클래스를 호출하는 것 뿐이니 직접 호출하면 그만
  - UIManager가 이미 'UI 입출력을 관리한다'는 정체성을 가지니, UIManager에서 리셋 버튼도 관리하게 하자
    - 이 경우는 클래스의 확고한 정체성을 기반으로 한 것이므로 문제가 되지 않는다

버튼 클릭 처리

- 버튼 클릭 시 함수 호출은 Unity 에디터에서도 할 수 있지만, 스크립트를 통해서도 할 수 있다
- 스크립트를 통한 버튼 클릭 처리가 유용한 이유: 처음부터 씬에 있는 오브젝트가 아닌, 동적 생성되는 오브젝트의 함수도 할당 가능

- 버튼 클릭 또한 일종의 이벤트로 처리된다
- 버튼의 Button 컴포넌트 이름을 button으로 저장했을 때,
  - 버튼 클릭 이벤트: button.onClick
  - 버튼 클릭 이벤트 구독: button.onClick.AddListener((함수));
  - 버튼 클릭 이벤트 구독 해제: button.onClick.RemoveListener((함수));

버튼 클릭 처리

- 설계 흐름


UIManager
버튼 클릭 이벤트
리스너 할당
UIManager
버튼 클릭 감지
GameManager
의 리셋 함수 호출
GameManager
스테이지 리셋

실습 5: 풀스택 개발

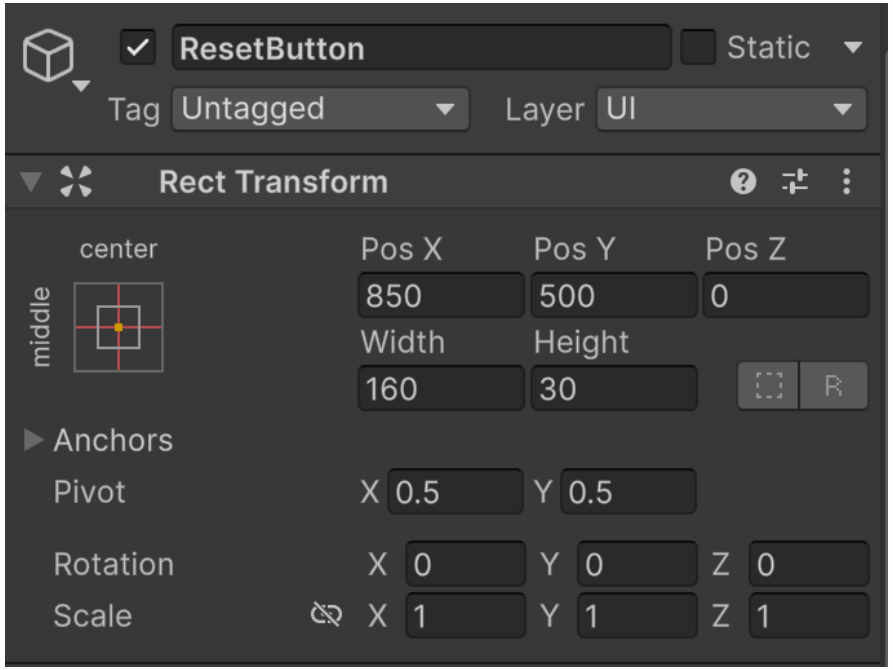
- 앞 장에서 나온 설계 흐름을 보고, 직접 리셋 버튼을 구현해보자

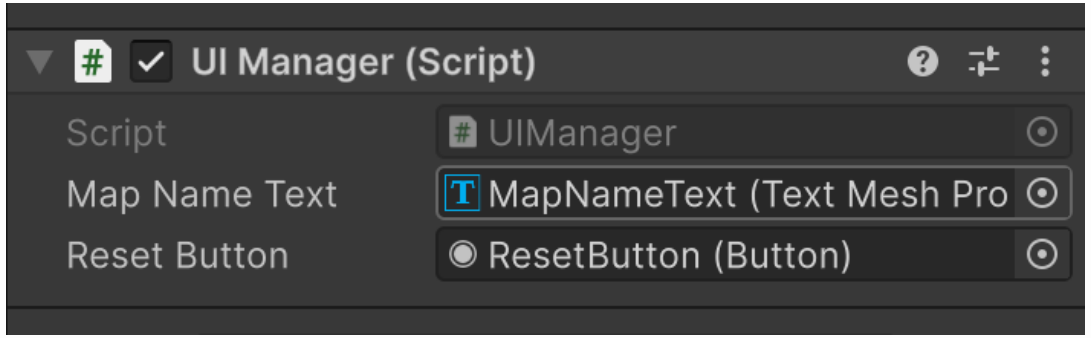
- 힌트
  - Button "컴포넌트"는 어떻게 받아야 할까?
    - 스크립트에서 Button 컴포넌트를 쓰려면 using UnityEngine.UI; 가 필요하다
  - AddListener는 이벤트에서의 += 처럼 쓰면 된다
  - 이벤트를 할당했으면 할당 해제도 잊지 말자

  - 절대 강의 자료 만들기가 귀찮아서 실습으로 때운 게 아니다

실습 5: 풀스택 개발

- 정답


ResetButton
Static
Tag
Untagged
Layer
UI
Rect Transform
center
middle
Pos X
Pos Y
Pos Z
850
500
0
Width
Height
160
30
Anchors
Pivot
X 0.5
Y 0.5
Rotation
X 0
Y 0
Z 0
Scale
X 1
Y 1
Z 1


UI Manager (Script)
Script
UIManager
Map Name Text
MapNameText (Text Mesh Pro
Reset Button
ResetButton (Button)

실습 5: 풀스택 개발

- 정답

- 코드는 Github Gist의 UIManager_1.cs 참조

```
using UnityEngine;
using UnityEngine.UI;

1 애셋 사용 위치
public class UIManager : MonoBehaviour
{
    [SerializeField] private TMPro.TextMeshProUGUI _mapNameText;
    [SerializeField] private Button _resetButton;  ResetButton (Button)

    이벤트 함수
    void Awake()
    {
        GameManager.Instance.OnMapLoaded += OnMapLoaded;
        _resetButton.onClick.AddListener(OnResetButtonClick);
    }

    이벤트 함수
    void OnDestroy()
    {
        GameManager.Instance.OnMapLoaded -= OnMapLoaded;
        _resetButton.onClick.RemoveListener(OnResetButtonClick);
    }

    2 사용 위치
    void OnMapLoaded(string mapName)
    {
        _mapNameText.text = mapName;
    }

    2 사용 위치
    void OnResetButtonClick()
    {
        GameManager.Instance.ResetMap();
    }
}
```

이번 주차 요약

- UI
- 디자인 패턴(싱글톤 패턴, 옵저버 패턴)
- 이벤트

- 다음 주차: 10/28